Computers and Chemical Engineering 139 (2020) 106844

Contents lists available at ScienceDirect

Computers and Chemical Engineering

journal homepage: www.elsevier.com/locate/compchemeng

Stochastic data-driven model predictive control using gaussian processes

Eric Bradford[a,*], Lars Imsland[a], Dongda Zhang[b,c], Ehecatl Antonio del Rio Chanona[b]

[a] Department of Engineering Cybernetics, Norwegian University of Science and Technology, Trondheim, Norway
[b] Centre for Process Systems Engineering (CPSE), Department of Chemical Engineering, Imperial College London, UK
[c] School of Chemical Engineering and Analytical Science, University of Manchester,Manchester, UK

ARTICLE INFO

Article history:
Received 6 December 2019
Revised 24 March 2020
Accepted 29 March 2020

Keywords:
Model-based nonlinear control
Uncertain dynamic systems
Machine learning
Probabilistic constraints
State space
Robust control

ABSTRACT

Nonlinear model predictive control (NMPC) is one of the few control methods that can handle multivariable nonlinear control systems with constraints. Gaussian processes (GPs) present a powerful tool to identify the required plant model and quantify the residual uncertainty of the plant-model mismatch. It is crucial to consider this uncertainty, since it may lead to worse control performance and constraint violations. In this paper we propose a new method to design a GP-based NMPC algorithm for finite horizon control problems. The method generates Monte Carlo samples of the GP offline for constraint tightening using back-offs. The tightened constraints then guarantee the satisfaction of chance constraints online. Advantages of our proposed approach over existing methods include fast online evaluation, consideration of closed-loop behaviour, and the possibility to alleviate conservativeness by considering both online learning and state dependency of the uncertainty. The algorithm is verified on a challenging semi-batch bioprocess case study.

1. Introduction

Model predictive control (MPC) describes an advanced control method that has found a wide range of applications in industry. MPC employs an explicit dynamic model of the plant to determine a finite sequence of control actions to take at each sampling time. The main advantage of MPC is its ability to deal with multivariate plants and process constraints explicitly (Maciejowski, 2002). Linear MPC is relatively mature and well-established in practice, however many systems display strong nonlinear behaviour motivating the use of nonlinear MPC (NMPC) (Allgöwer et al., 2004). NMPC is becoming progressively more popular due to the advancement of improved non-convex optimization algorithms (Biegler, 2010), in particular in chemical engineering (Biegler and Rawlings, 1991).

The performance of MPC is however greatly influenced by the accuracy of the plant model, which is one of the main reasons why MPC is not more widely used in industry (Lucia, 2014). The development of an accurate plant model has been cited to take up to 80% of the MPC commissioning effort (Sun et al., 2013). NMPC algorithms exploit various types of models, commonly developed by first principles or based on process mechanisms (Nagy et al., 2007b). Many mechanistic and empirical models are however often too complex to be used online and in addition have often high development costs. Alternatively, black-box identification models can be exploited instead, such as support vector machines (Xi et al., 2007), fuzzy models (Kavsek-Biasizzo et al., 1997), neural networks (NNs) (Piche et al., 2000), or Gaussian processes (GPs) (Kocijan et al., 2004). For example, recently in Wu et al. (2019c,b) recurrent NNs are utilised for an extensive NMPC approach with proofs on closed-loop state boundedness and convergence applied to a chemical reactor. In addition, in Wu et al. (2019a) the approach was extended updating the recurrent NNs online to further improve the effectiveness.

GPs are an interpolation technique developed by Krige (1951) that were popularized by the machine learning community (Rasmussen and Williams, 2006). While GPs have been predominantly used to model static nonlinearities, there are several works that apply GPs to model dynamic systems (Girard et al., 2003; Kocijan et al., 2005; Bradford et al., 2018b). GP predictions are given by a Gaussian distribution. The mean of this distribution can be viewed as a deterministic prediction, while

* Corresponding author.
E-mail addresses: eric.bradford@ntnu.no (E. Bradford), a.del-rio-chanona@imperial.ac.uk (E.A. del Rio Chanona).

https://doi.org/10.1016/j.compchemeng.2020.106844

the variance can be interpreted as a measure of uncertainty for this deterministic prediction. This uncertainty measure is generally difficult to obtain by nonlinear parametric models (Kocijan et al., 2004) and may in part explain the relative popularity of GPs. For control applications this uncertainty measure can be utilized to efficiently learn a dynamic model by exploring unknown regions or avoiding regions with too high uncertainty to improve robustness (Berkenkamp and Schoellig, 2015). So far, GPs have been exploited in a multitude of ways in the control community, including reinforcement learning (Deisenroth and Rasmussen, 2011), designing robust linear controllers (Umlauft et al., 2017), or adaptive control (Chowdhary et al., 2015). In particular, GPs have been shown to be an efficient approach to attain approximate plant models for NMPC.

The use of GPs for NMPC was first proposed in Murray-Smith et al. (2003), which updates a GP model online for reference tracking without constraints. In Kocijan et al. (2004); Kocijan and Murray-Smith (2005) the GP is instead identified offline and utilized online, in which the variance is constrained to prevent the NMPC from steering the dynamic system into regions with high uncertainty. A GP plant model is updated online in Klenske et al. (2016) and in Maciejowski and Yang (2013) to overcome unmodeled periodic errors or changes to the dynamic system after a fault has occurred respectively. GPs have been shown in Wang et al. (2016) to be an efficient means for disturbance forecasting for a linear stochastic MPC approach applied to a drinking water network. GPs have further been applied to approximate the mean and variance required in stochastic NMPC (Bradford and Imsland, 2018b). Grancharova et al. (2007) derived an explicit solution for GP-based NMPC. In Cao et al. (2017) a GP dynamic model is employed for the control of an unmanned quadrotor, while in Likar and Kocijan (2007) a GP dynamic model is exploited to control a gas-liquid separation process. While these and other works show the feasibility of GP-based MPC, there is a lack of efficient approaches to account for the uncertainty measure provided. Model uncertainty can lead to constraint violations and worse performance. To mitigate the effect of uncertainty on MPC, robust MPC (Bemporad and Morari, 1999) and stochastic MPC (Mesbah, 2016; Heirung et al., 2018) methods have been developed.

The majority of works for GP-based MPC indeed consider the uncertainty measure provided, however most proposed algorithms employ stochastic uncertainty propagation to achieve this, for example (Kocijan et al., 2004; Kocijan and Murray-Smith, 2005; Hewing et al., 2018; Cao et al., 2017; Grancharova et al., 2007; Wang et al., 2016). Hewing et al. (2019) give an overview of the various stochastic propagation techniques available. These approaches have some considerable disadvantages, which are:

- No known methods to exactly propagate stochastic uncertainties through GP models. Instead, only approximations are available relying on linearization or statistical moment-matching.
- Increased computational time of GP-based MPC due to the propagation approach itself.
- Most works consider only open-loop propagation of uncertainties, which is often prohibitively conservative due to open-loop growth of uncertainties.

Recently some papers have proposed different robust GP-based MPC algorithms. In Koller et al. (2018b) a NMPC algorithm is introduced based on propagating ellipsoidal sets using linearization, that provides closed-loop stability guarantees. This approach may however suffer from increased computational times, since the ellipsoidal sets are propagated online. Furthermore, the method may be relatively conservative due to the use of Lipschitz constants. Maiworm et al. (2018) propose the use of a robust MPC approach by bounding the one-step ahead error, while the determination of the required parameters seems to be relatively difficult. Soloperto et al. (2018) suggest a robust control approach for linear systems, in which the GP is used to represent unmodeled nonlinearities. The approach is shown to stabilize the linear system despite these uncertainties, which however may have no solution if the difference between the linearized system and the actual nonlinear system is too large.

In this paper we extend an algorithm first introduced in Bradford et al. (2019), for which the following extensions were made:

- Inclusion of uncertainty for the initial state.
- Adding additive disturbance noise to the problem definition.
- Accounting for state dependency on the GP noise.
- Improved algorithm to obtain the required back-offs using root-finding as opposed to the inverse CDF, which has superior convergence and leads to improved satisfaction of the required probability bounds.

The aim of this approach is to take into account the uncertainty given by a GP state space model for a NMPC *finite-horizon* control problem. Due to the issues using stochastic uncertainty propagation for the NMPC formulation as highlight previously, we base the NMPC only on cheap evaluations of the GP. This leads to considerably faster evaluation times with little effect on the performance. The proposed method utilizes explicit back-offs, which were recently proposed in Koller et al. (2018a); Paulson and Mesbah (2018) to account for stochastic uncertainties in NMPC. These methods generally rely on generating closed-loop Monte Carlo (MC) samples offline from the plant to attain the required back-off values. To obtain exact MC samples of the GP dynamic models we exploit results from Conti et al. (2009); Umlauft et al. (2018). There are several important advantages of this new method:

- Back-offs are attained using closed-loop simulations, therefore the issue of open-loop growth of uncertainties is avoided.
- Required computations are carried out offline, such that the online computational times are nearly unaffected.
- Independence of samples allows some probabilistic guarantees to be given.
- Explicit consideration of online learning and state dependency of the uncertainty to alleviate conservativeness.

The proposed method is a data-driven NMPC approach relying on black-box identification of a GP model from input/output data pairs. The required data may be obtained either by simulations from a high-fidelity model or by experiments using for example step tests. In the algorithm the state dependency of the uncertainty is accounted for by introducing a penalty term on the variance in the objective. This variance for GPs is a function of the states and hence leads to a trade-off between exploiting the GP model to optimize the objective and avoiding uncertain regions to reduce the spread of the trajectories. The algorithm proposed is aimed at *finite horizon* control problems, for which batch processes are a particularly important example. They are utilized in many different chemical engineering sectors due to their inherent flexibility to deal with variations in feedstock, product specifications, and market demand. Frequent highly nonlinear behaviour and unsteady-state operation of batch processes have led to the increased acceptance for advanced control solutions, such as NMPC (Nagy et al., 2007a). Works on batch process NMPC accounting for uncertainties include an extended and Unscented Kalman filter based algorithms for uncertainty propagation (Nagy and Braatz, 2003; Bradford and Imsland, 2018a), a NMPC algorithm using min-max successive linearization (Valappil and Georgakis, 2002), NMPC algorithms that employ PCEs to account for possible parametric uncertainties (Mesbah et al., 2014; Bradford and Imsland, 2019), and multi-stage NMPC (Lucia et al., 2013).

The paper is comprised of the following sections. In Section 2 the problem definition is given. In Section 3 a general outline of GPs is given including the sampling procedure used. Section 4 shows how the GPs can be exploited to solve the defined problem. In Section 5 the semi-batch bioprocess case study is described, while in Section 6 the results and discussions for this case study are given. Section 7 concludes the paper.

Notation

$\mathbb{N}$ and $\mathbb{R}$ represent the sets of natural numbers and real numbers respectively. The variable δ_{ij} denotes the Kronecker delta function, such that:

$$\delta_{ij} := \begin{cases} 1, & \text{if } i = j \\ 0, & \text{otherwise} \end{cases}$$

The notation $\text{diag}(a_0, a_1, \ldots, a_n)$ is used to represent the following diagonal matrix:

$$\text{diag}(a_0, a_1, \ldots, a_n) := \begin{bmatrix} a_0 & 0 & \ldots & 0 \\ 0 & a_1 & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \ldots & 0 & a_n \end{bmatrix}$$

We represent the Gaussian distribution with mean μ and covariance $\mathbf{\Sigma}$ as $\mathcal{N}(\mu, \mathbf{\Sigma})$. Further, $\phi \sim \mathcal{N}(\mu, \mathbf{\Sigma})$ denotes that the random variable ϕ follows a Gaussian distribution with mean μ and covariance $\mathbf{\Sigma}$.

The expected value of a random variable ϕ is denoted as:

$$\mathbb{E}[\phi] := \int_{\Omega} \phi dp_{\phi}$$

where p_{ϕ} the probability density function of ϕ over the sample space Ω.

Further, we define the indicator function and the probability measure of random variable ϕ as follows:

$$\mathbf{1}\{C \leq c\} := \begin{cases} 1, & \text{if } C \leq c \\ 0, & \text{otherwise} \end{cases}$$

$$\mathbb{P}\{\phi \in \mathcal{A}\} := \int_{\phi \in \mathcal{A}} \phi dp_{\phi}, \quad \mathbb{P}\{\phi \leq c\} := \mathbb{E}[\mathbf{1}\{\phi \leq c\}]$$

where $\mathcal{A}$ is a set defining an event on ϕ and $\mathbb{P}\{\phi \leq c\}$ is the probability that ϕ is less than or equal to c.

Lastly, we require the definition of the beta inverse cumulative distribution function (cdf) for a random variable ϕ. This function betainv(P, A, B) returns a value C of ϕ following a beta distribution with parameters A, B that has a probability of P to be less than or equal to C. The definition is as follows:

$$\text{betainv}(P, A, B) \in F_{\phi}^{-1}(P|A, B) = \{C : F_{\phi}(C|A, B) = P)\}$$

$$F_{\phi}(C|A, B) := \frac{1}{\mathcal{B}(A, B)} \int_0^C t^{A-1}(1-t)^{B-1} dt,$$

$$\mathcal{B}(A, B) = \int_0^1 t^{A-1}(1-t)^{B-1} dt$$

2. Problem definition

The dynamic system in this paper is given by a discrete-time nonlinear equation system with additive disturbance noise:

$$\mathbf{x}_{t+1} = \mathbf{f}(\mathbf{x}_t, \mathbf{u}_t) + \omega_t, \quad \mathbf{x}_0 \sim \mathcal{N}(\mu_{\mathbf{x}_0}, \mathbf{\Sigma}_{\mathbf{x}_0}) \tag{1}$$

where t is the discrete time, $\mathbf{x} \in \mathbb{R}^{n_\mathbf{x}}$ is the state, $\mathbf{u} \in \mathbb{R}^{n_\mathbf{u}}$ are the control inputs, $\mathbf{f} : \mathbb{R}^{n_\mathbf{x}} \times \mathbb{R}^{n_\mathbf{u}} \to \mathbb{R}^{n_\mathbf{x}}$ are nonlinear equations, and ω represents Gaussian distributed additive disturbance noise with zero mean and diagonal covariance matrix $\mathbf{\Sigma}_\omega$. The initial condition $\mathbf{x}_0$ is assumed to be Gaussian distributed with mean $\mu_{\mathbf{x}_0}$ and covariance matrix $\mathbf{\Sigma}_{\mathbf{x}_0}$.

We assume measurements of the states to be available with additive Gaussian noise expressed as:

$$\mathbf{y} = \mathbf{f}(\mathbf{x}, \mathbf{u}) + \nu \tag{2}$$

where $\mathbf{y} \in \mathbb{R}^{n_\mathbf{x}}$ is the measurement of $\mathbf{f}(\mathbf{x}, \mathbf{u})$ perturbed by additive Gaussian noise $\nu \sim \mathcal{N}(\mathbf{0}, \mathbf{\Sigma}_\nu)$ with zero mean and a diagonal covariance matrix $\mathbf{\Sigma}_\nu = \text{diag}(\sigma_{\nu_1}^2, \ldots, \sigma_{\nu_{n_\mathbf{x}}}^2)$.

The aim of the control problem is to minimize a finite-horizon cost function:

$$V_T(\mathbf{x}_0, \mathbf{U}) = \mathbb{E}\left[\sum_{t=0}^{T-1} \ell(\mathbf{x}_t, \mathbf{u}_t) + \ell_f(\mathbf{x}_T)\right] \tag{3}$$

where $T \in \mathbb{N}$ is the time horizon, $\mathbf{U} = [\mathbf{u}_0, \ldots, \mathbf{u}_{T-1}]^\top \in \mathbb{R}^{T \times n_\mathbf{u}}$ is a joint vector of T control inputs, $\ell : \mathbb{R}^{n_\mathbf{x}} \times \mathbb{R}^{n_\mathbf{u}} \to \mathbb{R}$ is the stage cost, and $\ell_f : \mathbb{R}^{n_\mathbf{x}} \to \mathbb{R}$ denotes the terminal cost.

The control problem is subject to hard constraints on the inputs:

$$\mathbf{u}_t \in \mathbb{U}_t \quad \forall t \in \{0, \ldots, T-1\} \tag{4}$$

The states are subject to a joint chance constraint that requires the satisfaction of a nonlinear constraint set up to a certain probability, which can be stated as:

$$\mathbb{P}\left\{\bigcap_{t=0}^{T} \{\mathbf{x}_t \in \mathbb{X}_t\}\right\} \geq 1 - \epsilon \tag{5a}$$

where $\mathbb{X}_t$ is defined as:

$$\mathbb{X}_t = \{\mathbf{x} \in \mathbb{R}^{n_\mathbf{x}} \mid g_j^{(t)}(\mathbf{x}) \leq 0, j = 1, \ldots, n_g\} \tag{5b}$$

The joint chance constraints are formulated such that the joint event over all $t \in \{0, \ldots, T\}$ of all $\mathbf{x}_t$ fulfilling the nonlinear constraint sets $\mathbb{X}_t$ has a probability greater than $1 - \epsilon$.

For convenience we define the tuple $\mathbf{z} = (\mathbf{x}, \mathbf{u}) \in \mathbb{R}^{n_\mathbf{z}}$ with joint dimension $n_\mathbf{z} = n_\mathbf{x} + n_\mathbf{u}$. The dynamic system in Eq. (1) is assumed to be unknown. Instead, we are only given a *finite number* of noisy measurements according to Eq. (2). The available data can then be denoted by the following two matrices:

$$\mathbf{Z} = [\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(N)}]^\top \in \mathbb{R}^{N \times n_\mathbf{z}} \tag{6a}$$

$$\mathbf{Y} = [\mathbf{y}^{(1)}, \ldots, \mathbf{y}^{(N)}]^\top \in \mathbb{R}^{N \times n_\mathbf{x}} \tag{6b}$$

where $\mathbf{z}^{(i)}$ represents the input of the i-th data point with corresponding noisy observation $\mathbf{y}^{(i)}$, N denotes the overall number of training data points, $\mathbf{Z}$ is a collection of input data, and the corresponding noisy observations are collected in $\mathbf{Y}$.

It should be noted that the uncertainty in this problem arises partially from the uncertain initial condition $\mathbf{x}_0$ and the additive disturbance noise ω. Most of the uncertainty however comes from the fact that we do not know $\mathbf{f}(\mathbf{x}, \mathbf{u})$ and are only given noisy observations of $\mathbf{f}(\mathbf{x}, \mathbf{u})$ instead. To solve this problem we train a GP to approximate $\mathbf{f}(\cdot)$ using the available data in Eq. (6). The GP methodology is introduced for this purpose in the next section. This GP then represents a distribution over possible functions $\mathbf{f}(\cdot)$ given the available data, which can be exploited to attain stochastic constraint satisfaction of the closed-loop system.

3. Gaussian processes

3.1. Regression

In this section we introduce the use of GPs to infer a latent function $f : \mathbb{R}^{n_\mathbf{z}} \to \mathbb{R}$ from noisy data. For a more complete

overview refer to Rasmussen and Williams (2006). Let the noisy observations y of $f(\cdot)$ be given by:

$$y = f(\mathbf{z}) + \nu \tag{7}$$

where $\mathbf{z} \in \mathbb{R}^{n_\mathbf{z}}$ is the argument of $f(\cdot)$ and y is a perturbed observation of $f(\mathbf{z})$ with additive Gaussian noise $\nu \sim \mathcal{N}(0, \sigma_\nu^2)$ with zero mean and variance σ_ν^2.

GPs can be considered a generalization of multivariate Gaussian distributions to describe a distribution over functions. A GP is fully specified by a mean function and a covariance function. The mean function represents the "average" shape of the function, while the covariance function specifies the covariance between any two function values. We write that a function $f(\cdot)$ is distributed as a GP with mean function $m(\cdot)$ and covariance function $k(\cdot, \cdot)$ as:

$$f(\cdot) \sim \mathcal{GP}(m(\cdot), k(\cdot, \cdot)) \tag{8}$$

The prior GP distribution is defined by the *choice* of the mean function and covariance function. In this study we apply a zero mean function and the squared-exponential (SE) covariance function defined as:

$$m(\mathbf{z}) := 0 \tag{9}$$

$$k(\mathbf{z}, \mathbf{z}') := \zeta^2 \exp\left(-\frac{1}{2}(\mathbf{z} - \mathbf{z}')^\mathsf{T} \mathbf{\Lambda}^{-2} (\mathbf{z} - \mathbf{z}')\right) \tag{10}$$

where $\mathbf{z}, \mathbf{z}' \in \mathbb{R}^{n_\mathbf{z}}$ are arbitrary inputs, ζ^2 denotes the covariance magnitude, and $\mathbf{\Lambda}^{-2} := \text{diag}(\lambda_1^{-2}, \ldots, \lambda_{n_\mathbf{z}}^{-2})$ is a scaling matrix.

Remark (Prior assumptions)**.** Note the zero mean assumption can be easily achieved by normalizing the data beforehand. The SE covariance function is smooth and stationary, such that choosing the SE covariance function assumes the latent function $f(\cdot)$ to be smooth and stationary as well. The algorithm presented in this work can be utilised using any covariance function. In the case of highly non-stationary functions it may be necessary to use non-stationary covariance functions (Sampson and Guttorp, 1992).

From the additive property of Gaussian distributions the measurements of $f(\cdot)$ also follow a GP accounting for measurement noise:

$$y \sim \mathcal{GP}(m(\mathbf{z}), k(\mathbf{z}, \mathbf{z}') + \sigma_\nu^2 \delta_{\mathbf{z}\mathbf{z}'}) \tag{11}$$

We denote the hyperparameters defining the prior jointly by $\mathbf{\Psi} := [\zeta, \lambda_1, \ldots, \lambda_{n_\mathbf{z}}, \sigma_\nu]^\mathsf{T}$, in which the variance σ_ν of the measurement noise is included in case it is unknown. Commonly the hyperparameters are unknown, such that they need to be inferred from the available data using for example maximum likelihood estimation (MLE).

Assume we are given N noisy function evaluations according to Eq. (7) denoted by $\mathbf{Y} := [y^{(1)}, \ldots, y^{(N)}]^\mathsf{T} \in \mathbb{R}^N$ as the result of the inputs given in $\mathbf{Z} = [\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(N)}]^\mathsf{T} \in \mathbb{R}^{N \times n_\mathbf{z}}$. According to the prior GP assumption, the data follows a multivariate Gaussian distribution:

$$\mathbf{Y} \sim \mathcal{N}(\mathbf{0}, \mathbf{\Sigma}_\mathbf{Y}) \tag{12}$$

where $[\mathbf{\Sigma}_\mathbf{Y}]_{ij} = k(\mathbf{z}^{(i)}, \mathbf{z}^{(j)}) + \sigma_\nu^2 \delta_{ij}$ for each pair $(i, j) \in \{1, \ldots, N\}^2$.

The log-likelihood of the observations is consequently given by (ignoring constant terms):

$$\mathcal{L}(\mathbf{\Psi}) := -\frac{1}{2}\log(\det(\mathbf{\Sigma}_\mathbf{Y})) - \frac{1}{2}\mathbf{Y}^\mathsf{T}\mathbf{\Sigma}_\mathbf{Y}^{-1}\mathbf{Y} \tag{13}$$

The MLE estimate of the hyperparameters $\mathbf{\Psi}$ is determined by maximizing Eq. (13). Once the hyperparameters are known, we need to determine the posterior GP distribution of the latent function $f(\cdot)$. From the prior GP assumption we know that the training data and the value of $f(\cdot)$ at an arbitrary input $\mathbf{z}$ follow a joint multivariate normal distribution:

$$\begin{bmatrix} \mathbf{Y} \\ f(\mathbf{z}) \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mathbf{0} \\ 0 \end{bmatrix}, \begin{bmatrix} \mathbf{\Sigma}_\mathbf{Y} & \mathbf{k}^\mathsf{T}(\mathbf{z}) \\ \mathbf{k}(\mathbf{z}) & k(\mathbf{z}, \mathbf{z}') \end{bmatrix}\right) \tag{14}$$

where $\mathbf{k}(\mathbf{z}) := [k(\mathbf{z}, \mathbf{z}^{(1)}), \ldots, k(\mathbf{z}, \mathbf{z}^{(N)})]^\mathsf{T}$.

The posterior Gaussian distribution of $f(\mathbf{z})$ given the data $(\mathbf{Z}, \mathbf{Y})$ can then be found by using the conditional distribution rule for multivariate normal distributions based on the joint normal distribution in Eq. (14), which leads to:

$$f(\mathbf{z})|\mathcal{D} \sim \mathcal{N}(\mu_f(\mathbf{z}; \mathcal{D}), \sigma_f(\mathbf{z}; \mathcal{D})) \tag{15a}$$

with

$$\mu_f(\mathbf{z}; \mathcal{D}) := \mathbf{k}^\mathsf{T}(\mathbf{z})\mathbf{\Sigma}_\mathbf{Y}^{-1}\mathbf{Y} \tag{15b}$$

$$\sigma_f^2(\mathbf{z}; \mathcal{D}) := \zeta^2 - \mathbf{k}^\mathsf{T}(\mathbf{z})\mathbf{\Sigma}_\mathbf{Y}^{-1}\mathbf{k}(\mathbf{z}) \tag{15c}$$

where $\mathcal{D} = (\mathbf{Z}, \mathbf{Y})$ denotes the training data available to obtain the posterior Gaussian distribution. The mean function $\mu_f(\mathbf{z}; \mathcal{D})$ in this context is the prediction of the GP at $\mathbf{z}$, while the variance function $\sigma_f^2(\mathbf{z}; \mathcal{D})$ is a measure of uncertainty.

In Fig. 1 we illustrate a prior GP in the top graph and the posterior GP in the bottom graph.

3.2. Recursive update

Often we are given a training dataset $\mathcal{D}$ initially to build a posterior GP and then obtain data points individually afterwards. For example in GP-based MPC we may build an initial GP offline following the procedure shown in Section 3.1, and then also update this model online as new data becomes available. In this work we keep the hyperparameters constant but update the mean function and variance function in Eqs. 15b–15c recursively given the new data points. Furthermore, the approach used for MC sampling of the GPs to be introduced in Section 3.4 requires recursive updates of this form as well.

Let the new dataset be given by $\mathcal{D}^+ = (\mathcal{D}, (\mathbf{z}^+, y^+))$, where $\mathcal{D}$ is the training data for the initial GP, while $\mathbf{z}^+$ is the new input and y^+ the new corresponding output measurement. Then we will refer to the updated mean function and variance function as:

$$\mu_f^+(\mathbf{z}; \mathcal{D}^+) := \mathbf{k}^{+\mathsf{T}}(\mathbf{z})\mathbf{\Sigma}_\mathbf{Y}^{+-1}\mathbf{Y}^+ \tag{16a}$$

$$\sigma_f^{2+}(\mathbf{z}; \mathcal{D}^+) := \zeta^2 - \mathbf{k}^{+\mathsf{T}}(\mathbf{z})\mathbf{\Sigma}_\mathbf{Y}^{+-1}\mathbf{k}^+(\mathbf{z}) \tag{16b}$$

The updated terms in Eqs. 16a–16b can be expressed as:

$$\mathbf{k}^+(\mathbf{z}) = [\mathbf{k}^\mathsf{T}(\mathbf{z}), k(\mathbf{z}, \mathbf{z}^+)]^\mathsf{T} \tag{17a}$$

$$\mathbf{Y}^+ = [\mathbf{Y}^\mathsf{T}, y^+]^\mathsf{T}, \quad \mathbf{Z}^+ = [\mathbf{Z}^\mathsf{T}, \mathbf{z}^+]^\mathsf{T} \tag{17b}$$

$$\mathbf{\Sigma}_\mathbf{Y}^{+-1} = \begin{bmatrix} \mathbf{\Sigma}_\mathbf{Y}\mathbf{k}^\mathsf{T}(\mathbf{z}^+) \\ \mathbf{k}(\mathbf{z}^+)k(\mathbf{z}^+, \mathbf{z}^+)(+\sigma_\nu^2) \end{bmatrix}^{-1} \tag{17c}$$

where $\mathbf{k}(\mathbf{z})$, $\mathbf{Z}$ and $\mathbf{Y}$ refer to quantities of the initial GP. The noise term σ_ν^2 in the lower diagonal is shown in brackets, since the new "measurement" y^+ may be noiseless as is the case for GP MC samples. In this case the noise term should not be added to the new diagonal element.

Note the updates $\mathbf{k}^+(\mathbf{z})$, $\mathbf{Z}^+$, and $\mathbf{Y}^+$ are trivial, however the update of the *inverse* covariance matrix $\mathbf{\Sigma}_\mathbf{Y}^{+-1}$ is more involved. In essence we require the inverse of the previous covariance matrix after adding a horizontal row and a vertical row to the covariance matrix of the initial GP, see Eq. (17c). For this process there are efficient formula available, one of which is introduced in Appendix A.

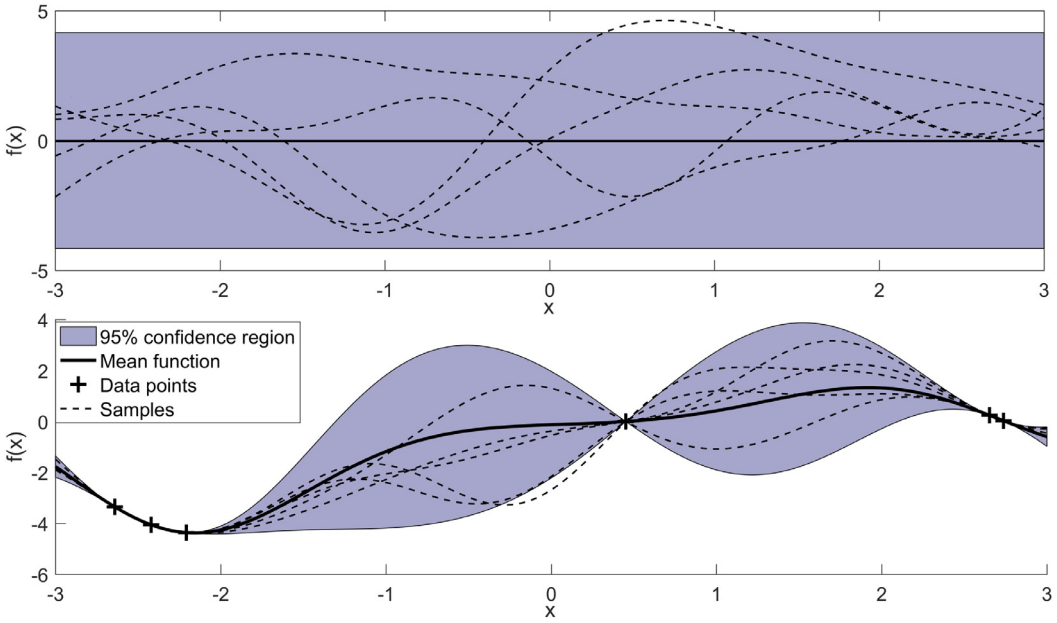

Fig. 1. Illustration of a GP of a 1-dimensional function perturbed by noise. On the top the prior of the GP is shown, while on the bottom the Gaussian process was fitted to several observations to obtain the posterior.

These take advantage of the fact that we already know the inverse covariance matrix $\mathbf{\Sigma}_{\mathbf{Y}}^{-1}$ of the initial GP. Once the update has been carried out, these terms then define the new initial GP. This update procedure is then repeated for the next measurement.

3.3. State space model

In this section we briefly show how the previously introduced GP methodology can be utilized to identify unknown state space models in the form of Eq. (1) based on the measurements (data) according to Eq. (2). GPs are commonly applied to model scalar functions with vector inputs as shown in Section 3.1. To extend this to the multi-input, multi-output case as required it is common to build a separate independent GP for each output dimension, see for example Deisenroth and Rasmussen (2011). Let the function in Eq. (1) be given by $\mathbf{f}(\mathbf{x},\mathbf{u}) := \mathbf{f}(\mathbf{z}) := [f_1(\mathbf{z}),\dots,f_{n_\mathbf{x}}(\mathbf{z})]^\mathsf{T}$. We aim to build a separate GP for each function $f_i(\mathbf{z})\ \forall i \in \{1,\dots,n_\mathbf{x}\}$ according to Section 3.1. For this purpose we are given observations $\mathbf{Y}_i = [y_i^{(1)},\dots,y_i^{(N)}]^\mathsf{T}\ \forall i \in \{1,\dots,n_\mathbf{x}\}$ and corresponding inputs $\mathbf{Z} = [\mathbf{z}^{(1)},\dots,\mathbf{z}^{(N)}]^\mathsf{T}$, where y_i refers to the i-th dimension of measurements obtained according to Eq. (2). Let $\mathbf{Y} = [\mathbf{Y}_1,\dots,\mathbf{Y}_{n_\mathbf{x}}]$ correspond to the overall measurements available. The posterior Gaussian distribution of $\mathbf{f}(\,\cdot\,)$ at an arbitrary input $\mathbf{z} = (\mathbf{x},\mathbf{u})$ is:

$$\mathbf{f}(\mathbf{z})|\mathcal{D} \sim \mathcal{N}(\mu_f(\mathbf{z};\mathcal{D}), \mathbf{\Sigma}_f(\mathbf{z};\mathcal{D})) \tag{18a}$$

with

$$\mu_f(\mathbf{z};\mathcal{D}) = [\mu_f(\mathbf{z};\mathcal{D}_1),\dots,\mu_f(\mathbf{z};\mathcal{D}_{n_\mathbf{x}})]^\mathsf{T} \tag{18b}$$

$$\mathbf{\Sigma}_f(\mathbf{z};\mathcal{D}) = \mathrm{diag}(\sigma_f^2(\mathbf{z};\mathcal{D}_1),\dots,\sigma_f^2(\mathbf{z};\mathcal{D}_{n_\mathbf{x}})) + \mathbf{\Sigma}_\omega \tag{18c}$$

where $\mu_f(\mathbf{z};\mathcal{D}_i)$ and $\sigma_f^2(\mathbf{z};\mathcal{D}_i)$ are the mean function and variance function built according to Section 3.1 with datasets $\mathcal{D}_i = (\mathbf{Z},\mathbf{Y}_i)\ \forall i \in \{1,\dots,n_\mathbf{x}\}$ with $\mathcal{D} = (\mathbf{Z},\mathbf{Y})$.

Remark (Additive disturbance noise)**.** Note the additive disturbance noise defined in Eq. (1) is simply added to the posterior covariance matrix due to the additive property of multivariate Gaussian distributions.

In addition, given an initial GP state space model built with a dataset $\mathcal{D}$ and a new data point $(\mathbf{z}^+,\mathbf{y}^+)$, we can update it recursively utilizing the method introduced in Section 3.2:

$$\mu_f^+(\mathbf{z};\mathcal{D}^+) = [\mu_f^+(\mathbf{z};\mathcal{D}_1^+),\dots,\mu_f^+(\mathbf{z};\mathcal{D}_{n_\mathbf{x}}^+)]^\mathsf{T} \tag{19a}$$

$$\mathbf{\Sigma}_f^+(\mathbf{z};\mathcal{D}^+) = \mathrm{diag}(\sigma_f^{2+}(\mathbf{z};\mathcal{D}_1^+),\dots,\sigma_f^{2+}(\mathbf{z};\mathcal{D}_{n_\mathbf{x}}^+)) + \mathbf{\Sigma}_\omega \tag{19b}$$

where $\mathcal{D}_i^+ = (\mathcal{D}_i,(\mathbf{z}^+,y_i^+))\ \forall i \in \{1,\dots,n_\mathbf{x}\}$ and $\mathcal{D}^+ = (\mathcal{D},(\mathbf{z}^+,\mathbf{y}^+))$

3.4. Monte carlo sampling

GPs are distribution over functions and hence their realizations yield *deterministic* functions, see for example the GP samples shown in Fig. 1. In this section we show how to attain independent samples of GP state space models over a *finite* time horizon. Generating a MC sample of a GP would require sampling an infinite dimensional stochastic process, while there is no known method to achieve this. Instead, approximate approaches have been applied such as spectral sampling (Bradford et al., 2018a; Quionero-Candela et al., 2010). Exact samples of GPs are however possible if the GP MC sample needs to be known at only a finite number of points, which is exactly the situation for state space models over a *finite* time horizon. This technique was first outlined in Conti et al. (2009) and has been employed in Umlauft et al. (2018) for the optimal design of linear controllers. We next outline how to obtain an exact sample of a GP state space model over a *finite* time horizon for an arbitrary feedback control policy.

Assume we are given a GP state space model as shown in Section 3.3 from the input-output dataset $\mathcal{D} = (\mathbf{Z},\mathbf{Y})$. The initial condition $\mathbf{x}_0$ is assumed to follow a known Gaussian distribution as defined in Eq. (1). A GP state space model represents a distribution over possible plant models, for which each realization will lead to a different state sequence. The aim of this section is therefore to show how to obtain a single independent sample of such a state sequence, which can then be repeated to obtain multiple independent MC samples of the GP. Let $\mathcal{X}^{(s)} = [\chi_0^{(s)},\chi_1^{(s)},\dots,\chi_T^{(s)}]^\mathsf{T}$ denote such a state sequence, where s denotes a particular GP realization and $\chi_i^{(s)}$ the realization of the state at discrete time i in

Algorithm 1 Gaussian process trajectory sampling.

Input : $\mu_{\mathbf{x}_0}$, $\mathbf{\Sigma}_{\mathbf{x}_0}$, $\mu_f(\mathbf{z};\mathcal{D})$, $\mathbf{\Sigma}_f(\mathbf{z};\mathcal{D})$, $\mathcal{D}$, T, $\kappa(\cdot)$

Initialize: Draw $\chi_0^{(s)}$ from $\mathbf{x}_0 \sim \mathcal{N}(\mu_{\mathbf{x}_0}, \mathbf{\Sigma}_{\mathbf{x}_0})$.

for *each sampling time* $t = 1, 2, \ldots, T$ **do**

1. Determine $u_{t-1}^{(s)} = \kappa(\chi_{t-1}^{(s)}, t-1)$
2. Draw $\chi_t^{(s)}$ from $\mathbf{x}_t \sim \mathcal{N}(\mu_f(z_{t-1}^{(s)};\mathcal{D}), \mathbf{\Sigma}_f(z_{t-1}^{(s)};\mathcal{D}))$, where $z_{t-1}^{(s)} = (\chi_{t-1}^{(s)}, u_{t-1}^{(s)})$.
3. Define $\mathcal{D}^+ := (\mathcal{D}, (z_{t-1}^{(s)}, \chi_t^{(s)}))$, where $\chi_t^{(s)}$ can be viewed as noiseless "measurements".
4. Update the dataset $\mathcal{D} := ([\mathbf{Z}^\top, z_{t-1}^{(s)\top}]^\top, [\mathbf{Y}, \chi_t^{(s)\top}]^\top)$.
5. Recursively update the GP mean function $\mu_f(\mathbf{z};\mathcal{D}) := \mu_f^+(\mathbf{z};\mathcal{D}^+)$ using Equation 19a.
6. Recursively update the GP covariance function $\mathbf{\Sigma}_f(\mathbf{z};\mathcal{D}) := \mathbf{\Sigma}_f^+(\mathbf{z};\mathcal{D}^+)$ using Equation 19b.

end

Output : State sequence $\mathcal{X}^{(s)} = [\chi_0^{(s)}, \chi_1^{(s)}, \ldots, \chi_T^{(s)}]^\top$ and control sequence $\mathcal{U}^{(s)} = [u_0^{(s)}, \ldots, u_{T-1}^{(s)}]^\top$

the sequence of MC sample s. The control inputs at discrete time i for MC sample s are denoted by $u_i^{(s)}$. The corresponding joint input of $\chi_i^{(s)}$ is represented by $z_i^{(s)} = (\chi_i^{(s)}, u_i^{(s)})$. We assume the control inputs to be the result of a feedback control policy, which we represent as $\kappa : \mathbb{R}^{n_\mathbf{x}} \times \mathbb{R} \to \mathbb{R}^{n_\mathbf{u}}$. The control actions $u_i^{(s)}$ are then given as follows:

$$u_i^{(s)} = \kappa\left(\chi_i^{(s)}, i\right) \tag{20}$$

where i is the current discrete time. Note the control policy depends on the discrete time directly, since it is a finite horizon control policy.

Consequently, the control actions over the finite time horizon T are a function of $\mathcal{X}^{(s)}$ and denoted jointly as $\mathcal{U}^{(s)} = [u_0^{(s)}, \ldots, u_{T-1}^{(s)}]^\top = [\kappa(\chi_0^{(s)}, 0), \ldots, \kappa(\chi_{T-1}^{(s)}, T-1)]^\top$. Note these control inputs are different for each MC sample s due to feedback.

We start by sampling the Gaussian distribution of the initial state $\mathbf{x}_0 \sim \mathcal{N}(\mu_{\mathbf{x}_0}, \mathbf{\Sigma}_{\mathbf{x}_0})$ to obtain the realization $\chi_0^{(s)}$. The posterior Gaussian distribution of the next state in the sequence $\mathbf{x}_1$ is subsequently given by the GP of $\mathbf{f}(\cdot)$ as defined in Eq. (18) dependent on $\chi_0^{(s)}$:

$$\mathbf{x}_1 = \mathbf{f}(z_0) \sim \mathcal{N}(\mu_f(z_0;\mathcal{D}), \mathbf{\Sigma}_f(z_0;\mathcal{D})) \tag{21}$$

The realization of $\mathbf{x}_1$ is obtained by sampling the above normal distribution, which we will denote as $\chi_1^{(s)}$. To obtain the next state in the sequence $\chi_2^{(s)}$ we need to first condition on $\chi_1^{(s)}$, since this is part of this sampled function path. This requires to treat $\chi_1^{(s)}$ similarly to a new training point, however without observation noise (i.e. no σ_v^2 is added to the kernel evaluation $k(z_0, z_0)$) and without changing the hyperparameters. Note that if the sampled function were to return to the same input it would lead to the exact same output, since it is conditioned on a noiseless output. This shows that the sampled function is deterministic, since it is the result of sampling. Next we draw $\chi_2^{(s)}$ according to the posterior Gaussian distribution obtained from adding the previously sampled data point to the training dataset $\mathcal{D}$ as a noiseless observation. This sample is then again added to the training dataset as a noiseless observation, from which the GP is updated and the next state is drawn. This process is repeated until the required time horizon T has been reached. In this paper we consider a finite time horizon control problem and hence the GP state space model should for, moderate time horizons, not become too computationally expensive, since at most T new data-points are added. Nonetheless, for large time horizons this could become a problem and approximate sampling approaches, such as spectral sampling should then be considered instead (Bradford et al., 2018a; Quionero-Candela et al., 2010).

This sampling approach is summarized in Algorithm 1 below and is illustrated in Fig. 2. Each GP MC sample is defined by a state and corresponding control action sequence. Note that this gives us a single MC sample and subsequently needs to be repeated multiple times to obtain multiple realizations.

Lastly, we define a *nominal* trajectory by setting all samples in Algorithm 1 to their mean values. Let $\bar{\mathcal{X}} = [\bar{\chi}_0, \bar{\chi}_1, \ldots, \bar{\chi}_T]^\top$ refer to this *nominal* state sequence and $\bar{\mathcal{U}} = [\bar{u}_0, \ldots, \bar{u}_{T-1}]^\top$ to the corresponding *nominal* control sequence, where $\bar{\chi}_i$ and $\bar{u}_i$ are the values of the states and control inputs of the *nominal* trajectory at discrete time i respectively. Therefore by definition, $\bar{\chi}_0 = \mu_{\mathbf{x}_0}$, $\bar{u}_{t-1} = \kappa(\bar{\chi}_{t-1}, t-1)$, and $\bar{\chi}_t = \mu_f(\bar{z}_{t-1};\mathcal{D})$, where $\bar{z}_{t-1} = (\bar{\chi}_{t-1}, \bar{u}_{t-1})$. Note updating $\mu_f(\cdot;\mathcal{D})$ with mean values has no effect.

In Section 4 a NMPC formulation is introduced, which uses the mean function of the GP as the prediction model. The control actions from this NMPC algorithm then define the control policy in Eq. (20), while the MC samples represent possible plant responses. This is then exploited to tune the GP NMPC algorithm to attain the desired behaviour.

4. Solution approach

From the input-output dataset $\mathcal{D} = (\mathbf{Z}, \mathbf{Y})$ defined in Eq. (6), we fit a GP state space model as outlined in Section 3.3. The aim is to solve the problem defined in Section 2 based on this GP state space model. In this context the GP represents a distribution over possible plant models for the process given the available dataset. In Section 3.4 we have shown how to create a sample of this plant model over a finite time horizon T, which each lead to different state sequences and corresponding control sequences based on a control policy. In this paper we aim to design a NMPC algorithm based on the GP that acts as this control policy. The MC samples are utilized to tune the NMPC formulation by adjusting so-called back-offs to tighten the constraints and attain the closed-loop probabilistic constraint satisfaction. We next state the GP NMPC formulation, which is based on the tightened constraint set and predictions from the mean function $\mu_f(\cdot;\mathcal{D})$ and covariance function $\mathbf{\Sigma}_f(\cdot;\mathcal{D})$.

4.1. Finite-horizon gaussian process model predictive control formulation

In this section we define the NMPC OCP based on the GP *nominal* model given the dataset $\mathcal{D} = (\mathbf{Z}, \mathbf{Y})$. For the GP NMPC formulation the initial state $\mathbf{x}$ at each sampling time is assumed to be measured or estimated and propagated forward in time exploiting the GP mean function. The predicted states are then used to optimize the objective subject to the tightened constraints. Let the corresponding optimization problem be denoted as $P_T\left(\mu_f(\cdot;\mathcal{D}), \mathbf{\Sigma}_f(\cdot;\mathcal{D}); \mathbf{x}, t\right)$ for the current *known* state $\mathbf{x}$ at discrete time t based on the mean function $\mu_f(\cdot;\mathcal{D})$ and covariance

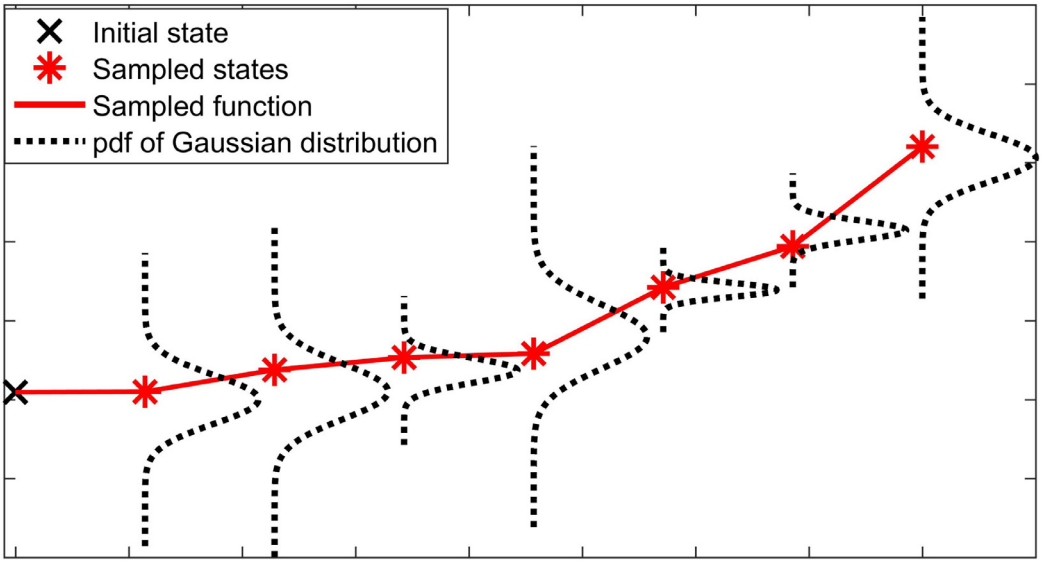

Fig. 2. Illustration of GP sampling scheme for a 1 dimensional function.

function $\mathbf{\Sigma}_f(\cdot;\mathcal{D})$:

$$\begin{aligned}
&\underset{\hat{\mathbf{U}}_{t:T-1}}{\text{minimize}} \quad \hat{V}_T(\mathbf{x},t,\hat{\mathbf{U}}_{t:T-1}) \\
&\quad = \sum_{k=t+1}^{T-1}\left[\ell(\hat{\mathbf{x}}_k,\hat{\mathbf{u}}_k)+\eta_k tr\big(\mathbf{\Sigma}_f(\hat{\mathbf{x}}_k;\mathcal{D})\big)\right]+\ell_f(\hat{\mathbf{x}}_T) \\
&\text{subject to:} \\
&\hat{\mathbf{x}}_{k+1}=\mu_f(\hat{\mathbf{z}}_k;\mathcal{D}), \quad \hat{\mathbf{z}}_k=(\hat{\mathbf{x}}_k,\hat{\mathbf{u}}_k) \qquad \forall k\in\{t,\dots,T-1\} \\
&\hat{\mathbf{x}}_{k+1}\in\overline{\mathbb{X}}_{k+1}, \quad \hat{\mathbf{u}}_k\in\mathbb{U}_k \qquad \forall k\in\{t,\dots,T-1\} \\
&\hat{\mathbf{x}}_t=\mathbf{x}
\end{aligned} \tag{22}$$

where $\hat{\mathbf{x}}$, $\hat{\mathbf{u}}$, and $\hat{V}_T(\cdot)$ refers to the states, control inputs, and control objective of the MPC formulation, $\hat{\mathbf{U}}_{t:T-1}=[\hat{\mathbf{u}}_t,\dots,\hat{\mathbf{u}}_{T-1}]^\mathsf{T}$, η_k are weighting factors to penalize uncertainty, and $\overline{\mathbb{X}}_k$ is a tightened constraint set denoted by: $\overline{\mathbb{X}}_k=\{\mathbf{x}\in\mathbb{R}^{n_x}\mid g_j^{(k)}(\mathbf{x})+b_j^{(k)}\leq 0,\ j=1,\dots,n_g\}$. The variables $b_j^{(k)}$ represent so-called back-offs, which tighten the original constraints $\mathbb{X}_t$ defined in Eq. (5).

Remark (Objective in expectation)**.** It should be noted that the above objective in Eq. (22) does not exactly optimize the objective in Eq. (3), since it is difficult to obtain the expectation of a nonlinear function. Approximations of this can be found in Hewing et al. (2019), however these generally are considerably more expensive and often only lead to marginally improved performance.

Remark (Scaling for state dependency factors)**.** Note we have opted for scalar scaling factors η_k to account for state dependency. For this to work reliably it is therefore necessary to normalize the data to ensure that all data has approximately the same magnitude, for example normalizing the data to have zero mean and unit variance.

The NMPC algorithm solves $P_T\big(\mu_f(\cdot;\mathcal{D}),\mathbf{\Sigma}_f(\cdot;\mathcal{D});\mathbf{x}_t,t\big)$ at each sampling time t given the current state $\mathbf{x}_t$ to obtain an optimal control sequence:

$$\begin{aligned}
&\hat{\mathbf{U}}^*_{t:T-1}\big(\mu_f(\cdot;\mathcal{D}),\mathbf{\Sigma}_f(\cdot;\mathcal{D});\mathbf{x}_t,t\big) \\
&=\big[\hat{\mathbf{u}}^*_t\big(\mu_f(\cdot;\mathcal{D}),\mathbf{\Sigma}_f(\cdot;\mathcal{D});\mathbf{x}_t,t\big), \\
&\dots,\hat{\mathbf{u}}^*_{T-1}\big(\mu_f(\cdot;\mathcal{D}),\mathbf{\Sigma}_f(\cdot;\mathcal{D});\mathbf{x}_t,t\big)\big]^\mathsf{T}
\end{aligned} \tag{23}$$

Only the first optimal control action is applied to the plant at time t before the same optimization problem is solved at time $t+1$ with a new state measurement $\mathbf{x}_{t+1}$. This procedure implicitly defines the following feedback control law, which needs to be repeatedly solved for each new measurement $\mathbf{x}_t$:

$$\kappa\big(\mu_f(\cdot;\mathcal{D}),\mathbf{\Sigma}_f(\cdot;\mathcal{D});\mathbf{x}_t,t\big)=\hat{\mathbf{u}}^*_t\big(\mu_f(\cdot;\mathcal{D}),\mathbf{\Sigma}_f(\cdot;\mathcal{D});\mathbf{x}_t,t\big) \tag{24}$$

It is explicitly denoted that the control actions depend on the GP model used. There are several important variations of the GP NMPC control policy. Firstly, it may seem reasonable to update the mean and covariance function using the previous state measurement and corresponding input by applying the recursive update rules introduced in Section 3.2. We will refer to this as *learning*. This may however lead to a more expensive and less reliable NMPC algorithm.

Secondly, the algorithm may want to avoid regions in which there is great uncertainty due to sparsity of data. This can be achieved by assigning some of the η_k with non-zero values to penalize the algorithm moving into these regions with high variance. This explicitly takes advantage of the state dependency of the noise covariance function and is hence referred as *state dependent*. It should be noted that evaluation of the covariance function is computationally expensive. It was determined that setting only η_{t+1} to a non-zero value is often sufficient due to the continued feedback update, i.e. penalizing only the variance for the one-step ahead prediction at time t. These variations can be summarized as follows:

- *Learning*: Update the mean and variance function of the GP using the previous state measurement and the known corresponding input.
- *State dependent*: Set some η_k not equal to zero, which will lead to the NMPC algorithm trying to find a path that has less variance and hence exploiting the state dependent nature of the uncertainty.
- *Non learning*: Keep the mean and variance function the same throughout the run.
- *Non state dependent*: Set all η_k to zero and hence ignoring the possible state dependency of the uncertainty.

Remark (Full state feedback)**.** Note in the control algorithm we have assumed full state feedback, i.e. it is assumed that the full state can be measured without noise. This assumption can be dropped if required by introducing a suitable observer and introduced in the closed-loop simulations to account for this additional uncertainty.

4.2. Probabilistic guarantees

In this section we illustrate how to obtain probabilistic guarantees for the joint chance constraint introduced in Section 2 in Eq. (5) based on independent samples of the GP plant model. For convenience we define a single-variate random variable $C(\cdot)$ representing the satisfaction of the joint chance constraint (Curtis et al., 2018):

$$C(\mathbf{X}) = \inf_{(j,t)\in\{1,\dots,n_g\}\times\{0,\dots,T\}} g_j^{(t)}(\mathbf{x}_t) \tag{25a}$$

$$\mathbb{P}\{C(\mathbf{X}) \le 0\} = \mathbb{P}\left\{\bigcap_{t=0}^{T}\{\mathbf{x}_t \in \mathbb{X}_t\}\right\} \tag{25b}$$

where $\mathbf{X} = [\mathbf{x}_0, \dots, \mathbf{x}_T]^\mathsf{T}$ defines a state sequence, and $\mathbb{X}_t = \{\mathbf{x} \in \mathbb{R}^{n_\mathbf{x}} \mid g_j^{(t)}(\mathbf{x}) \le 0,\ j = 1, \dots, n_g\}$.

The probability in Eq. (25) is intractable, however a good non-parametric approximation is often achieved utilizing the so-called empirical cumulative distribution function (ecdf). We define the cdf to be approximated as follows:

$$F_{C(\mathbf{X})}(c) = \mathbb{P}\{C(\mathbf{X}) \le c\} \tag{26}$$

Assuming we are given S independent and identically distributed MC samples of $\mathbf{X}$ and hence of $C(\mathbf{X})$, the ecdf estimate of the true cdf in Eq. (26) is given by:

$$F_{C(\mathbf{X})}(c) \approx \hat{F}_{C(\mathbf{X})}(c) = \frac{1}{S}\sum_{s=1}^{S} \mathbf{1}\{C(\mathcal{X}^{(s)}) \le c\} \tag{27}$$

where $\mathcal{X}^{(s)}$ is the s-th MC sample and $\hat{F}_{C(\mathbf{X})}(c)$ is the ecdf approximation of the true cdf $F_{C(\mathbf{X})}(c)$.

The quality of the approximation in Eq. (27) strongly depends on the number of samples used and it is therefore desirable to quantify the residual uncertainty of the sample approximation. This problem has been studied to a great extent in the statistics literature (Clopper and Pearson, 1934). In addition, there are several works applying these results for chance constrained optimization, see for example Alamo et al. (2015). The main result applied in this study is given below in Theorem 1.

Theorem 1 (Confidence interval for empirical cumulative distribution function)**.** *Assume we are given a value of the ecdf, $\hat{\beta} = \hat{F}_{C(\mathbf{X})}(c)$, as defined in Eq. (27) based on S independent samples of $C(\mathbf{X})$, then the true value of the cdf, $\beta = F_{C(\mathbf{X})}(c)$, as defined in Eq. (26) has the following lower and upper confidence bounds:*

$$\mathbb{P}\left\{\beta \ge \hat{\beta}_{lb}\right\} \ge 1-\alpha, \quad \hat{\beta}_{lb} = \text{betainv}\left(\alpha, S\hat{\beta}, S - S\hat{\beta} + 1\right), \tag{28a}$$

$$\mathbb{P}\left\{\beta \le \hat{\beta}_{ub}\right\} \ge 1-\alpha, \quad \hat{\beta}_{ub} = \text{betainv}\left(1-\alpha, S\hat{\beta} + 1, S - S\hat{\beta}\right). \tag{28b}$$

Proof. The proof uses standard results in statistics and can be found in Clopper and Pearson (1934); Streif et al. (2014). The proof relies on the following observations. Firstly, $\mathbf{1}\{C(\mathcal{X}^{(s)}) \le c\}$ for a *fixed value* of c describes a Bernoulli random variable, in which either $C(\mathcal{X}^{(s)})$ exceeds c and takes the value 0 or otherwise takes the value 1 with probability $F_{C(\mathbf{X})}(c)$. Secondly, the ecdf describes the number of *successes* of S realizations of these Bernoulli random variables divided by the total number of samples and hence follows a binomial distribution, i.e. $\hat{F}_{C(\mathbf{X})}(c) \sim \frac{1}{N_S}\text{Bin}(S, F_{C(\mathbf{X})}(c))$. The confidence bound for the ecdf can consequently be determined from the Binomial cdf. This method was first introduced by Clopper and Pearson (1934) as "*exact confidence intervals*". Due to the close relationship between beta distributions and binomial distributions, a simplified expression can be obtained using beta distributions instead, which leads to the theorem shown (Streif et al., 2014). ■

In other words the probability of β exceeding the value $\hat{\beta}_{ub}$ has a probability of α and the probability of β being less than or equal to $\hat{\beta}_{lb}$ has also a probability of α. In particular, $\hat{\beta}_{lb}$ for small α represents a conservative lower bound on the true probability β. An illustration of the confidence bound for the ecdf is shown in Fig. 3. In general more samples will lead to a tighter confidence bound as expected. The theorem provides a lower bound $\hat{\beta}_{lb}$ that accounts for the statistical error due to the finite sample estimate made, i.e. it gives us a conservative value that is less than or equal to the true probability of feasibility with a confidence level of $1-\alpha$.

We assume we are given S *independent* samples of the trajectory $\mathbf{X}$ generated according to Section 3.4. Let the approximate ecdf be given by $\hat{\beta} = \hat{F}_{C(\mathbf{X})}(0)$ according to Eq. (27), and $\hat{\beta}_{lb}$ the corresponding lower bound according to Theorem 1 with confidence level $1-\alpha$. From this the following Corollary follows:

Corollary 1 (Feasibility probability)**.** *Assuming the GP representation of the plant model to be a correct description of the uncertainty of the system and given a value of the ecdf $\hat{\beta} = \hat{F}_{C(\mathbf{X})}(0)$, as defined in Eq. (27) based on S independent samples and a corresponding lower bound $\hat{\beta}_{lb} \ge 1-\epsilon$ with a confidence level of $1-\alpha$, then the original chance constraint in Eq. (5) holds true with a probability of at least $1-\alpha$.*

Proof. The GP MC sample described in Section 3.4 is exact and therefore each sample of the GP plant state space model leads to independent state trajectories $\mathbf{X}$ according to the GP distribution. From S such samples a valid lower bound $\hat{\beta}_{lb}$ to the true cdf value β can be determined from Theorem 1 with a confidence level of $1-\alpha$. If $\hat{\beta}_{lb}$ is greater than or equal to $1-\epsilon$, then the following probabilistic bound holds on the true cdf value β according to Theorem 1: $\mathbb{P}\left\{\beta \ge \hat{\beta}_{lb} \ge 1-\epsilon\right\} \ge 1-\alpha$, which in other words means that $\beta = \mathbb{P}\{C(\mathbf{X}) \le 0\} \ge 1-\epsilon$ with a probability of at least $1-\alpha$. ■

4.3. Determining back-off constraints

In this section we describe how to determine the required back-off values to tighten the constraints of the GP NMPC algorithm in Eq. (22). The aim is to choose these values to obtain probabilistic guarantees for the state constraints defined in Eq. (5) despite not knowing the exact dynamics. The GP provides a nominal model for the GP NMPC formulation in Eq. (22) using the mean function and a distribution of possible plant models given the initial dataset in Eq. (6). This distribution is exploited to attain different possible plant model realizations to simulate the closed-loop response of the GP NMPC algorithm and then uses the response values to tighten the constraints, such that the original constraint set is satisfied with a high probability according to Eq. (5).

We have shown how to obtain the closed-loop trajectory of a control policy according to the realization of a plant model GP distribution in Section 3.4. To this we apply the GP NMPC control policy defined in Eq. (24). We propose to utilize S independent MC samples of the GP distribution generated according to Section 3.4, which then in turn describe S different possible plant models with corresponding state and control trajectories. The goal now is to adjust the back-offs, such that the S different state trajectories adhere the original constraint set for all but a few samples to attain the required probability of constraint satisfaction. Let $\mathcal{X}^{(s)} = [\chi_0^{(s)}, \dots, \chi_T^{(s)}]^\mathsf{T}$ refer to the state trajectory of sample s.

The update rule for adjusting the back-offs is based on two steps: First, we define an approximate constraint set, which is then

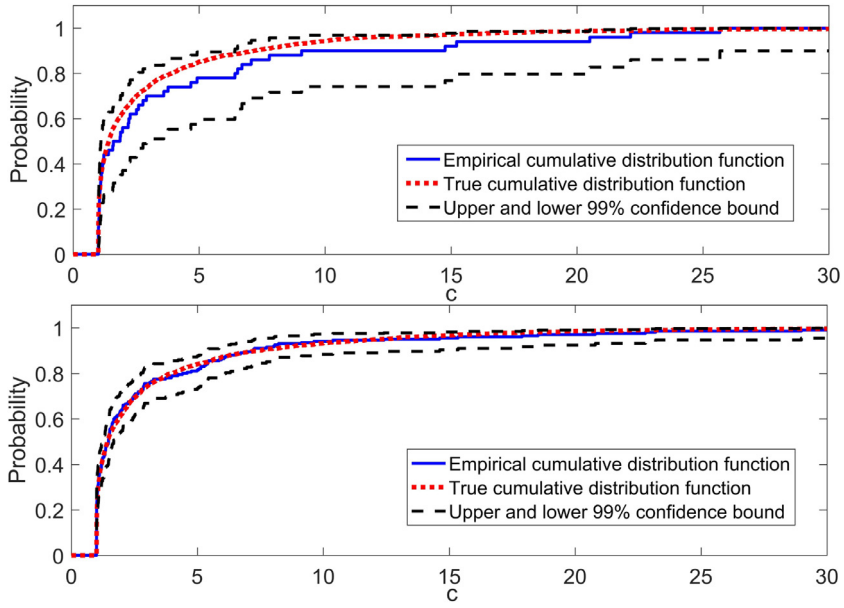

Fig. 3. Illustration of the cdf confidence bound of $\alpha = 0.01$ for sample sizes of $S = 50$ (top) and $S = 200$ (bottom).

adjusted by a constant factor to obtain the required constraint satisfaction using the ecdf of the joint chance constraint in Eq. (30). The approximate constraint set in essence needs to reflect the difference in constraint values for the nominal model of the MPC and the realizations of the GP plant model. We first set all the back-off values to zero and run S MC samples of the GP plant model. As defined in Section 3.4, let $\bar{x}_t$ refer to the states according to the *nominal* trajectory with the back-offs set to zero as well. Now assume we aim to obtain back-off values that imply the satisfaction of the following individual chance constraints:

$$g_j^{(t)}(\bar{x}_t) + b_j^{(t)} = 0 \Rightarrow \mathbb{P}\left\{g_j^{(t)}(x_t) \leq 0\right\} \geq 1 - \delta \tag{29}$$

where δ is a tuning parameter and should be set to a reasonably low value.

The rule in other words aims to find approximate back-offs for the nominal predictions $\bar{x}_t$ utilized in the MPC formulation in Eq. (22), such that the chance constraints holds for any possible GP plant model MC sample with a probability of $1 - \delta$. The parameter δ in this case is a tuning parameter to obtain the initial back-off values. Note the considered individual chance constraint in Eq. (29) is only there to obtain the *initial* constraint set and is unrelated to the joint chance constraint in Eq. (5), which we fulfil by adjusting this approximate constraint set using a root-finding algorithm.

To accomplish this we define the following ecdf based on the S MC samples available:

$$\hat{F}_{g_j^{(t)}}(0) = \frac{1}{S}\sum_{s=1}^{S} \mathbf{1}\{g_j^{(t)}(x_t^{(s)}) \leq 0\} \tag{30}$$

where $\hat{F}_{g_j^{(t)}}(0)$ is a sample approximation of the chance constraint given in Eq. (29) on the RHS.

In Paulson and Mesbah (2018) it is proposed to employ the inverse ecdf to approximately fulfill the requirement given in Eq. (29) using the S MC samples available. The back-offs can then be stated as follows:

$$\tilde{b}_j^{(t)} = \hat{F}^{-1}_{g_j^{(t)}}(1-\delta) - g_j^{(t)}(\bar{x}_t) \quad \forall (j,t) \in \{1,\dots,n_g^{(t)}\} \times \{1,\dots,T\} \tag{31}$$

where $\hat{F}^{-1}_{g_j^{(t)}}$ is the inverse of the ecdf defined in Eq. (30) and $\tilde{b}_j^{(t)}$ refers to these initial back-off values. Note the inverse of an ecdf is given by the quantile function of the discrete S values with probability δ. This gives us the initial back-off values as required for the first step. Note that both the nominal trajectory $\bar{x}_t$ and the GP realizations depend on the back-off values, which is however ignored since we are only interested in obtaining some reasonable initial values. In the next step these back-off values are further adjusted using a constant *back-off* factor γ. The new back-offs are then defined as:

$$b_j^{(t)} = \gamma \tilde{b}_j^{(t)} \quad \forall (j,t) \in \{1,\dots,n_g^{(t)}\} \times \{1,\dots,T\} \tag{32}$$

We aim to change γ until the lower bound of the ecdf $\hat{\beta}_{lb}$ as defined in the previous section for the joint chance constraint is equal to $1 - \epsilon$ in Eq. (5). This is a root finding problem, in which γ is adjusted until $\hat{\beta}_{lb}$ reaches the required value:

$$h(\gamma) = \hat{\beta}_{lb}(\gamma) - (1 - \epsilon) \tag{33}$$

where the aim is to determine a value of γ, such that $h(\gamma)$ is approximately zero.

To attain the required γ we use the so-called *bisection method* (Beers, 2007). This method determines the root of a function in an interval a_γ and b_γ, where $h(a_\gamma)$ and $h(b_\gamma)$ have opposite signs. In our case this is relatively easy. Setting the value of γ too low returns generally a negative value of $h(\gamma)$ due to the constraint violations using low back-offs, while setting it too high leads to positive values leading to a too conservative solution. Note we generally set the initial a_γ to zero since this corresponds to the S MC samples used to determine $\tilde{b}_j^{(t)}$. The *bisection method* consists of repeatably bisecting the interval, in which the root is contained. The overall algorithm to determine the required back-offs in n_b back-off iterations is summarized below as Algorithm 2. The output of the algorithm are the required back-offs with the corresponding lower bound on the probability of satisfying the state chance constraint. Note for *learning* = *true* the mean and covariance function of the GP are recursively updated utilizing the same procedure as for the update of the GP plant model MC sample.

Remark (Conservativeness of chance constraint)**.** Note to adjust the back-offs we use the ecdf, which does account for the *true*

Algorithm 2 Back-off iterative updates.

Input : $\mu_{\mathbf{x}_0}$, $\mathbf{\Sigma}_{\mathbf{x}_0}$, $\mu_f(\mathbf{z};\mathcal{D})$, $\mathbf{\Sigma}_f(\mathbf{z};\mathcal{D})$, $\mathcal{D}$, T, $V_T(\mathbf{x},t,\hat{\mathbf{U}}_{t:T-1})$, $\mathbb{X}_t$, $\mathbb{U}_t$, ϵ, α, δ, learning, S, n_b

Initialize: Set all $b_j^{(t)}=0$ and δ to some reasonable value, set $a_\gamma=0$ and b_γ to some reasonably high value, such that $b_\gamma-(1-\epsilon)$ has a positive sign. Define $\mathcal{D}_0=\mathcal{D}$ as the initial dataset.

for n_b *back-off iterations* **do**
- **if** $n_b>0$ **then**
  - $c_\gamma := (a_\gamma+b_\gamma)/2$
  - $b_j^{(t)} := c_\gamma \tilde{b}_j^{(t)} \quad (j,t)\in\{1,\dots,n_g^{(t)}\}\times\{1,\dots,T\}$
- **for** *each MC sample* $s=1,2,\dots,S$ **do**
  - $\mathcal{D}:=\mathcal{D}_0$
  - Draw $\chi_0^{(s)}$ from $\mathbf{x}_0\sim\mathcal{N}(\mu_{\mathbf{x}_0},\mathbf{\Sigma}_{\mathbf{x}_0})$
  - **for** *each sampling time* $t=1,2,\dots,T$ **do**
    - **if** *learning* $=$ *true* **then**
      - 1. Determine $u_{t-1}^{(s)}=\kappa(\mu_f(\cdot;\mathcal{D}),\mathbf{\Sigma}_f(\cdot;\mathcal{D});\mathbf{x}_t,t)$.
    - **else**
      - 1. Determine $u_{t-1}^{(s)}=\kappa(\mu_f(\cdot;\mathcal{D}_0),\mathbf{\Sigma}_f(\cdot;\mathcal{D}_0);\mathbf{x}_t,t)$.
    - **end**
    - 2. Draw $\chi_t^{(s)}$ from $\mathbf{x}_t\sim\mathcal{N}(\mu_f(z_{t-1}^{(s)};\mathcal{D}),\mathbf{\Sigma}_f(z_{t-1}^{(s)};\mathcal{D}))$, where $z_{t-1}^{(s)}=(\chi_{t-1}^{(s)},u_{t-1}^{(s)})$.
    - 3. Define $\mathcal{D}^+:=(\mathcal{D},(z_{t-1}^{(s)},\chi_t^{(s)}))$, where $\chi_t^{(s)}$ can be viewed as noiseless "measurements".
    - 4. Update the dataset $\mathcal{D}:=([\mathbf{Z}^\top,z_{t-1}^{(s)\top}]^\top,[\mathbf{Y},\chi_t^{(s)\top}]^\top)$.
    - 5. Recursively update the GP mean function $\mu_f(\mathbf{z};\mathcal{D}):=\mu_f^+(\mathbf{z};\mathcal{D}^+)$ using Equation 19a.
    - 6. Recursively update the GP covariance function $\mathbf{\Sigma}_f(\mathbf{z};\mathcal{D}):=\mathbf{\Sigma}_f^+(\mathbf{z};\mathcal{D}^+)$ using Equation 19b.
    - 7. Define $\boldsymbol{\mathcal{X}}^{(s)}=[\chi_0^{(s)},\chi_1^{(s)},\dots,\chi_T^{(s)}]^\top$ and $\boldsymbol{\mathcal{U}}^{(s)}=[u_0^{(s)},\dots,u_{T-1}^{(s)}]^\top$.
  - **end**
- **end**
- $\hat{\beta}:=\hat{F}_{C(\boldsymbol{\mathcal{X}}^{(s)})}(0)=\frac{1}{S}\sum_{s=1}^{S}\mathbf{1}\{C(\boldsymbol{\mathcal{X}}^{(s)})\le 0\}$
- $\hat{\beta}_{lb}:=1-\text{betainv}\left(\alpha,S+1-S\hat{\beta},S\hat{\beta}\right)$ **if** $nb=0$ **then**
  - Let $\tilde{b}_j^{(t)}=\hat{F}^{-1}_{g_j^{(t)}}(\delta)-g_j^{(t)}(\bar{\chi}_t)\quad\forall(j,t)\in\{1,\dots,n_g^{(t)}\}\times\{1,\dots,T\}$
  - $\hat{\beta}_{lb}^{a_\gamma}:=\hat{\beta}_{lb}-(1-\epsilon)$
- **else**
  - $\hat{\beta}_{lb}^{c_\gamma}:=\hat{\beta}_{lb}-(1-\epsilon)$
  - **if** $sign(\hat{\beta}_{lb}^{c_\gamma})=sign(\hat{\beta}_{lb}^{a_\gamma})$ **then**
    - $a_\gamma:=c_\gamma$
    - $\hat{\beta}_{lb}^{a_\gamma}:=\hat{\beta}_{lb}^{c_\gamma}$
  - **else**
    - $b_\gamma:=c_\gamma$
  - **end**
- **end**

end

Output : $b_j^{(t)}\quad\forall(j,t)\in\{1,\dots,n_g^{(t)}\}\times\{1,\dots,T\},\hat{\beta}_{lb}$

shape of the underlying probability distribution. This avoids the problem that is often faced in stochastic optimization utilizing Chebyshev's inequality to robustly approximate chance constraints, which is often excessively conservative (Paulson and Mesbah, 2019).

4.4. Algorithm

The overall algorithm proposed in this paper is summarized in this section. Firstly, the problem to be solved needs to be defined as outlined in Section 2. Thereafter, it needs to be decided if the GP NMPC should *learn* online or exploit the state dependency of the uncertainty as shown in Section 4. Once the GP NMPC has been formulated the back-offs are determined by running closed-loop simulations of the defined problem as shown in Section 4.3. Lastly, these back-offs then give us the tightened constraint set required for the GP NMPC *online*. This GP NMPC is then run online solving the problem initially outlined. An overall summary can be found in Algorithm 3.

Note the back-offs could be also updated online making use of the new initial conditions and updated prediction model in the case of "learning", which would lead to overall less conservativeness (Paulson and Mesbah, 2018). This would however require to carry-out the *offline* calculations online, which is expensive, and not computationally desirable.

5. Case study

The case study utilized in this paper deals with the photo-production of phycocyanin synthesized by cyanobacterium *Arthrospira platensis*. Phycocyanin is a high-value bioproduct and its biological function is to enhance the photosynthetic efficiency of cyanobacteria and red algae. It has been considered as a valuable compound because of its applications as a natural colorant to

Algorithm 3 Back-off GP NMPC.

Offline Computations

1. Build GP state-space model from data-set $\mathcal{D} = (\mathbf{Z}, \mathbf{Y})$ and additive disturbance $\mathbf{\Sigma}_\omega$.
2. Choose time horizon T, initial condition mean $\boldsymbol{\mu}_{\mathbf{x}_0}$ and covariance $\mathbf{\Sigma}_{\mathbf{x}_0}$, stage costs ℓ and ℓ_f, state dependent factor η_t, constraint sets $\mathbb{X}_t, \mathbb{U}_t \ \forall t \in \{1, \dots, T\}$, chance constraint probability ϵ, ecdf confidence α, tuning parameter δ, decide if *learning* should be carried out, the number of back-off iterations n_b and the number of Monte Carlo simulations S to estimate the back-offs.
3. Determine explicit back-off constraints using Algorithm 2.
4. Check final probabilistic value $\hat{\beta}_{lb}$ from Algorithm 2 if it is close enough to ϵ.

Online Computations

for $t = 0, \dots, T-1$ **do**

1. Solve the MPC problem in Equation 22 with the tightened constraint set from the *Offline Computations*.
2. Apply the first control input of the optimal solution to the real plant.
3. Measure the state $\mathbf{x}_t$ and update the GP plant model for learning GP NMPC.

end

replace other toxic synthetic pigments in both food and cosmetic production. Furthermore, it has shown great promise for the pharmaceutical industry because of its unique antioxidant, neuroprotective, and anti-inflammatory properties. Using a simplified dynamic model we verify our GP NMPC algorithm by operating this process using a limited dataset. The GP NMPC problem is formulated with an economic objective aiming to directly maximize the bioproduct concentration of the final batch subject to two path constraints and one terminal constraint.

5.1. Semi-batch bioreactor model

The simplified dynamic system consists of three ODEs describing the evolution of the concentration of biomass, nitrate, and bioproduct. The dynamic model is based on the Monod kinetics, which describes microorganism growth in nutrient sufficient cultures, where intracellular nutrient concentration is kept constant because of the rapid replenishment. We assume a fixed volume fed-batch. Control inputs are given by the light intensity (I) in μmol.m^{-2}.s^{-1} and nitrate inflow rate (F_N) in mg.L^{-1}.h^{-1} (del Rio-Chanona et al., 2015). To capture the effects of light intensity on microalgae growth and bioproduction (photolimitation, photosaturation, and photoinhibition phenomena) the Aiba model is used (Aiba, 1982). The balance equations are given as follows:

$$\frac{dC_X}{dt} = u_m \cdot \frac{I}{I + k_s + \frac{I^2}{k_i}} \cdot C_X \cdot \frac{C_N}{C_N + K_N} - u_d \cdot C_X, C_X(0) = C_{X0} \quad (34a)$$

$$\frac{dC_N}{dt} = -Y_{\frac{N}{X}} \cdot u_m \cdot \frac{I}{I + k_s + \frac{I^2}{k_i}} \cdot C_X \cdot \frac{C_N}{C_N + K_N} + F_N, C_N(0) = C_{N0} \quad (34b)$$

$$\frac{dC_{q_c}}{dt} = k_m \cdot \frac{I}{I + k_{sq} + \frac{I^2}{k_{iq}}} \cdot C_X - \frac{k_d C_{q_c}}{C_N + K_{Np}}, C_{q_c}(0) = C_{q_c 0} \quad (34c)$$

where C_X is the biomass concentration in g/L, C_N is the nitrate concentration in mg/L, and C_{q_c} is the phycocyanin (bioproduct) concentration in the culture in mg/L. The corresponding state vector and control vector are given by $\mathbf{x} = [C_X, C_N, C_{q_c}]^\mathsf{T}$ and $\mathbf{u} = [I, F_N]^\mathsf{T}$ respectively. The initial condition is denoted as $\mathbf{x}_0 = [C_{X0}, C_{N0}, C_{q_c 0}]^\mathsf{T}$. The missing parameter values can be found in Table 1.

5.2. Problem set-up

The time horizon T was set to 12 with an overall batch time of 240h, and consequently the sampling time is 20h. Based on the dynamic system in Eq. (34) we define the objective and the constraints according to the general problem definition in Section 2. The measurement noise matrix $\mathbf{\Sigma}_\nu$ and disturbance noise matrix $\mathbf{\Sigma}_\omega$ were set to:

$$\begin{aligned} \mathbf{\Sigma}_\nu &= \mathrm{diag}(4 \times 10^{-4}, 0.1, 1 \times 10^{-8}), \\ \mathbf{\Sigma}_\omega &= \mathrm{diag}(4 \times 10^{-4}, 0.1, 1 \times 10^{-8}) \end{aligned} \quad (35)$$

The mean $\boldsymbol{\mu}_{\mathbf{x}_0}$ and covariance $\mathbf{\Sigma}_{\mathbf{x}_0}$ of the initial condition are given by:

$$\boldsymbol{\mu}_{\mathbf{x}_0} = [1., 150, 0.]^\mathsf{T}, \quad \mathbf{\Sigma}_{\mathbf{x}_0} = \mathrm{diag}(1 \times 10^{-3}, 22.5, 0) \quad (36)$$

The control algorithm aims to maximize the amount of bioproduct produced C_{q_c} with a penalty on the change of control actions. The corresponding stage and terminal cost can be stated as follows:

$$\ell(\mathbf{x}_t, \mathbf{u}_t) = \mathbf{\Delta}_{\mathbf{u}_t}^\mathsf{T} \mathbf{R} \mathbf{\Delta}_{\mathbf{u}_t} \quad (37a)$$

$$\ell_f(\mathbf{x}_T) = -C_{q_c T} \quad (37b)$$

where $\mathbf{\Delta}_{\mathbf{u}_t} = \mathbf{u}_t - \mathbf{u}_{t-1}$ and $\mathbf{R} = \mathrm{diag}(3.125 \times 10^{-8}, 3.125 \times 10^{-6})$. The overall objective is then defined by Eq. (3).

For the case of state dependency we set all η_i to zero except η_0, see Equation 22. The value of η_0 was set to 15. Note that for these factors to work properly it is important to normalize the data as we did in this case study.

There are two path constraints in the problem. The amount of nitrate is constrained to remain below 800 mg/L, while the ratio of bioproduct to biomass may not exceed 11.0 mg/g for high density biomass cultivation. These constraints can be stated as:

$$g_1^{(t)}(\mathbf{x}_t) = C_{Nt} - 800 \leq 0 \qquad \forall t \in \{0, \dots, T\} \quad (38a)$$

Table 1
Parameter values for ordinary differential equation system in Eq. (34).

Parameter	Value	Units
u_m	0.0572	h^{-1}
u_d	0.0	h^{-1}
K_N	393.1	mg.L^{-1}
$Y_{\frac{N}{X}}$	504.5	mg.g^{-1}
k_m	0.00016	mg.g^{-1}.h^{-1}
k_d	0.281	h^{-1}
k_s	178.9	μmol.m^{-2}.s^{-1}
k_i	447.1	μmol.m^{-2}.s^{-1}
k_{sq}	23.51	μmol.m^{-2}.s^{-1}
k_{iq}	800.0	μmol.m^{-2}.s^{-1}
K_{Np}	16.89	mg.L^{-g1}

Table 2
The mean of the back-off values for the nitrate concentration constraints g_1/g_3 and the ratio of bioproduct to biomass constraint g_2 from the final back-off iteration.

Algorithm variation	Mean back-off g_1/g_3 (mg/L)	Mean back-off g_2 (mg/L)
GP NMPC 50	205.8	0.022
GP NMPC 60	34.2	0.008
GP NMPC 100	38.8	0.003
GP NMPC learning 50	54.0	0.007
GP NMPC 50 SD	23.3	0.002
GP NMPC 50 NSD	36.2	0.004

$$g_2^{(t)}(\mathbf{x}_t) = C_{q_c t} - 0.011 C_{Xt} \leq 0 \qquad \forall t \in \{0, \dots, T\} \tag{38b}$$

Lastly, there is a terminal constraint on nitrate to reach a final concentration of below 150 mg/L:

$$g_3^{(T)}(\mathbf{x}_T) = C_{NT} - 150 \leq 0, \quad g_3^{(t)}(\mathbf{x}_T) = 0 \; \forall t \in \{0, \dots, T-1\} \tag{39}$$

The maximum probability for violating the joint chance constraint was set to $\epsilon = 0.1$. The control inputs light intensity and nitrate inflow rate are constrained as follows:

$$120 \leq I_t \leq 400 \qquad \forall t \in \{0, \dots, T\} \tag{40a}$$

$$0 \leq F_{Nt} \leq 40 \qquad \forall t \in \{0, \dots, T\} \tag{40b}$$

For the back-off iterations we employed $S = 1000$ MC iterations with the initial back-offs computed according to Eq. (31) with $\delta = 0.1$ and $\alpha = 0.01$. The maximum number of back-off iterations for the bisection algorithm was set to $n_b = 16$.

5.3. Implementation details and initial dataset generation

The optimization problem for the GP NMPC in Eq. (22) is solved using Casadi (Andersson et al., 2019) to obtain the gradients of the problem using automatic differentiation in conjunction with IPOPT (Wächter and Biegler, 2006). The "real" plant model was simulated using IDAS (Hindmarsh et al., 2005). In the next section, different variations of the proposed algorithm are presented, for which two different type of datasets were collected. For the first type of dataset we designed the entire input data matrix **Z** according to a Sobol sequence (Sobol, 2001) in the range $\mathbf{z}_i \in [0, 20] \times [50, 800] \times [0, 0.18] \times [120, 400] \times [0, 40]$. The ranges were chosen for the data to cover the expected operating region. The corresponding outputs **Y** were then obtained from the IDAS simulation of the system perturbed by Gaussian noise as defined in the problem setup. In the second approach only the control inputs were set according to the Sobol sequence in the range $\mathbf{u}_i \in [120, 400] \times [0, 40]$ and the corresponding states **Y** were obtained from the trajectories of the "real" system perturbed by noise using samples of the initial condition and the time horizon as defined in the problem setup based on these control inputs. The system was simulated in "open-loop" using these control actions, i.e. without any feedback controller present. For both datasets the input data **Z** and output data **Y** are normalized to zero mean and a standard deviation of one. The reason we use two different types of datasets is to highlight the advantages of accounting for state dependency in two of the algorithm variations. In the first dataset the data is relatively evenly distributed and hence considering the state dependency of the uncertainty to avoid regions with high data sparsity has essentially no effect, while in the second approach there are clearly defined trajectories that can be followed by accounting for state dependency.

6. Results and discussions

In this section we present and discuss the results from the case study described in the previous section. For comparison purposes we compare six different variations of the proposed GP NMPC approach, which are as follows:

- GP NMPC 50, 60, 100: GP NMPC approach without learning and without taking into account state dependency for dataset sizes of 50, 60, and 100 points using the first type of dataset.
- GP NMPC learning 50: GP NMPC approach with learning and without state dependency for a dataset size of 50 points, which will be compared to the above case of 50 data points without learning. The first type dataset is utilized.
- GP NMPC SD/NSD 50: GP NMPC approach with and without accounting for the state dependency for a dataset size of 50 points employing the second type of dataset.

In addition, we compare the approaches to a nominal NMPC algorithm based on the GP model to show the importance of employing back-offs to prevent constraint violations, i.e. we run the GP NMPC on the "real" plant model, while setting the back-offs to zero. The results of the outlined runs are summarized in Figs. 6–12, and in Tables 2–3. In Figs. 6–7 we show the evolution of the back-off factor and the probability of constraint satisfaction $\hat{\beta}_{lb}$ over the 16 back-off iterations from Algorithm 2. The next two Figs. 4–5 show the 1000 MC trajectories of the constraints with a line to highlight the nominal prediction of the GP NMPC. Next the GP NMPC was applied to the "real" plant with back-offs from the final iteration and without back-offs referred to as *nominal* as shown in Figs. 8–9. Fig. 10 shows the probability density function of the objective values obtained from the "real" plant, where in the figure larger objective values correspond to better objective values. Fig. 11 shows representative control trajectories for GP NMPC 50, 60, 100 compared with the optimal trajectory obtained from solving the OCP of the "real" plant ignoring uncertainties. Fig. 12 shows the back-off values for the nitrate constraints g_1 and g_2 for GP NMPC 50 and GP NMPC 50 learning. Lastly, Table 2 shows the mean values for the back-offs averaged over time for the final back-off iteration, while Table 3 shows the attained probability of satisfaction $\hat{\beta}_{lb}$ together with the average computational times for solving a

Table 3
Lower bound on the probability of satisfying the joint constraint $\hat{\beta}_{lb}$, the approximate satisfaction probability from the final simulation $\hat{\beta}$, average computational times to solve a single optimal control problem (OCP) for the GP NMPC, and the average computational time required to complete one back-off iteration.

Algorithm variation	Probability $\hat{\beta}_{lb}$	Probability $\hat{\beta}$	OCP time (ms)	Back-off iteration time (s)
GP NMPC 50	0.99	1.00	65	782
GP NMPC 60	0.89	0.91	54	753
GP NMPC 100	0.91	0.93	135	1626
GP NMPC learning 50	0.91	0.93	69	825
GP NMPC 50 SD	0.89	0.91	48	574
GP NMPC 50 NSD	0.91	0.93	49	584

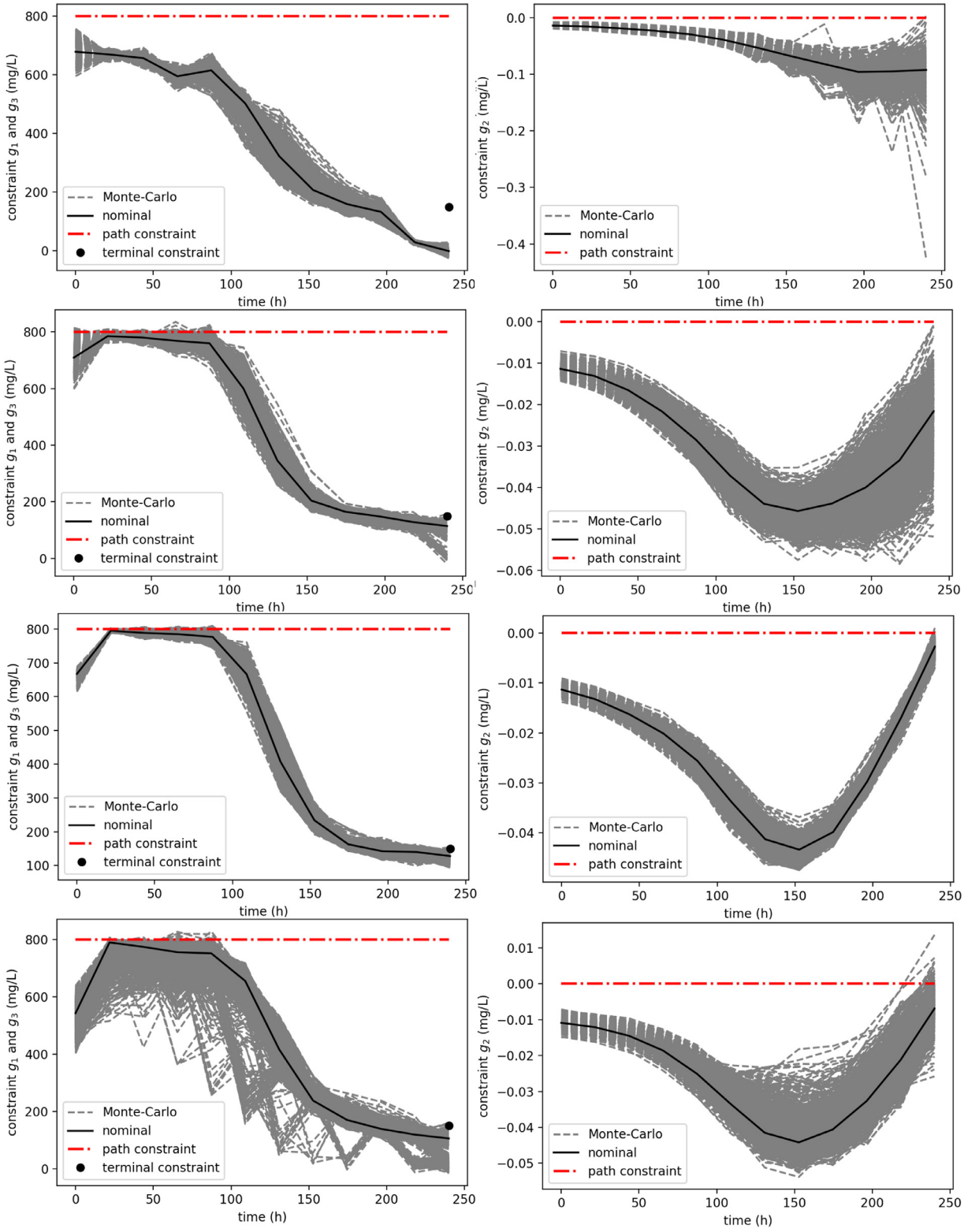

Fig. 4. The 1000 MC trajectories at the final back-off iteration of the nitrate concentration constraints g_1 and g_3 (LHS) and the ratio of bioproduct to biomass constraint g_2 (RHS) for from top to bottom GP NMPC 50, 60, 100 and GP NMPC 50 learning.

single GP NMPC optimization problem. We can draw the following conclusions from these results:

- Figs. 6–7 and Table 2 show that apart from GP NMPC 50 the other variations reach the required $\hat{\beta}_{lb}$ and hence successfully converge to a reasonable back-off factor. For these, as expected, a low back-off value leads to too low $\hat{\beta}_{lb}$ values near zero, while too high back-off values lead to too high $\hat{\beta}_{lb}$ values. The value of $\hat{\beta}_{lb}$ does vary by $\pm$ 0.01 even on convergence, which is due to the randomness of the MC samples. Nonetheless, since $\hat{\beta}_{lb}$ is a sample robust value, it is high enough if at least 0.9 is

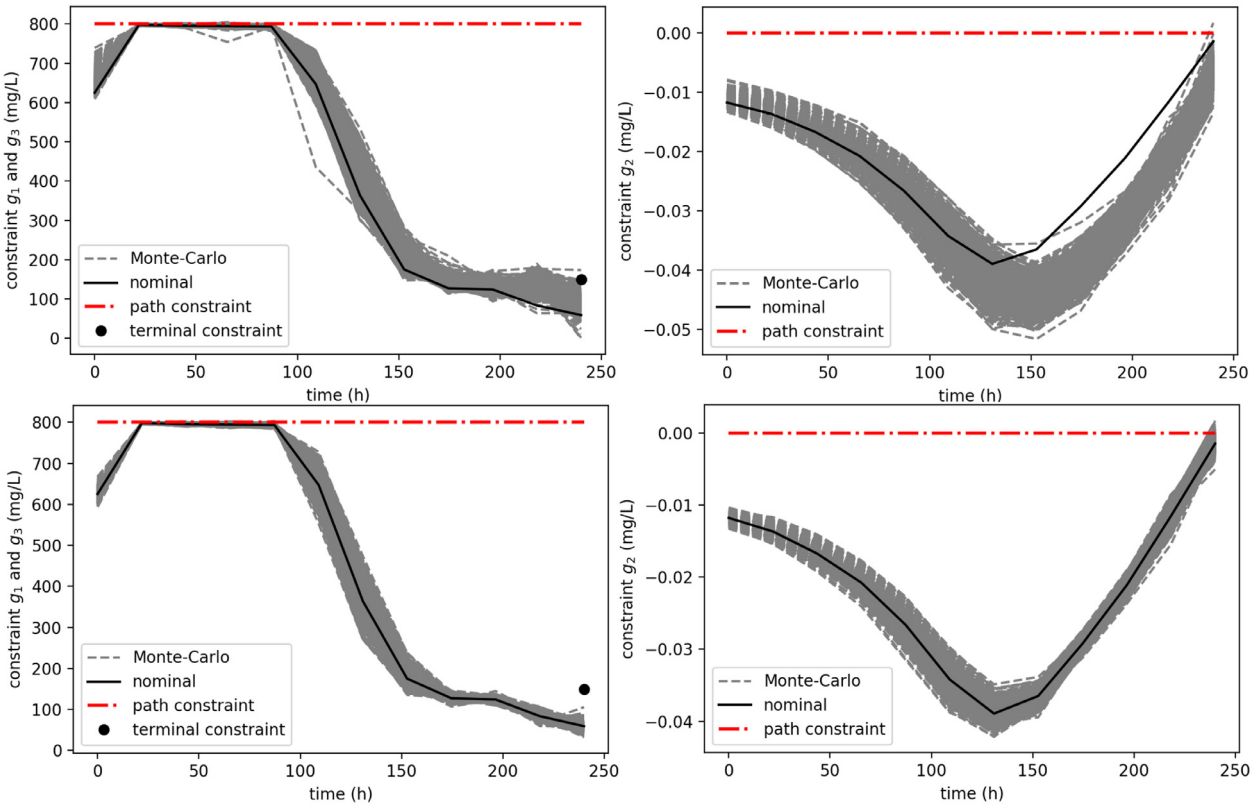

Fig. 5. The 1000 MC trajectories at the final back-off iteration of the nitrate concentration constraints g_1 and g_3 (LHS) and the ratio of bioproduct to biomass constraint g_2 (RHS) for GP NMPC NSD 50 (top) and GP NMPC SD 50 (bottom).

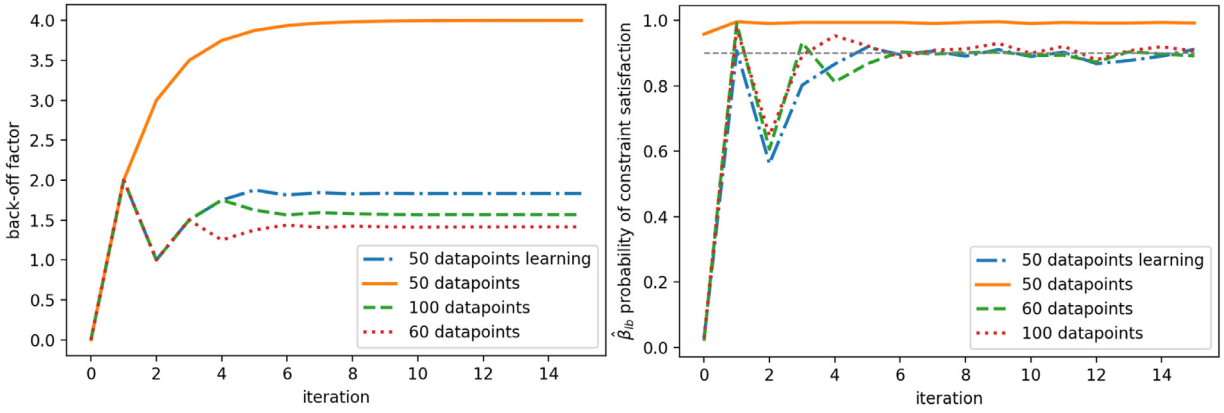

Fig. 6. Plots of evolution of the back-off factor and the probability of constraint satisfaction $\hat{\beta}_{lb}$ over the 16 back-off iterations for GP NMPC 50, 60, 100, and GP NMPC learning 50.

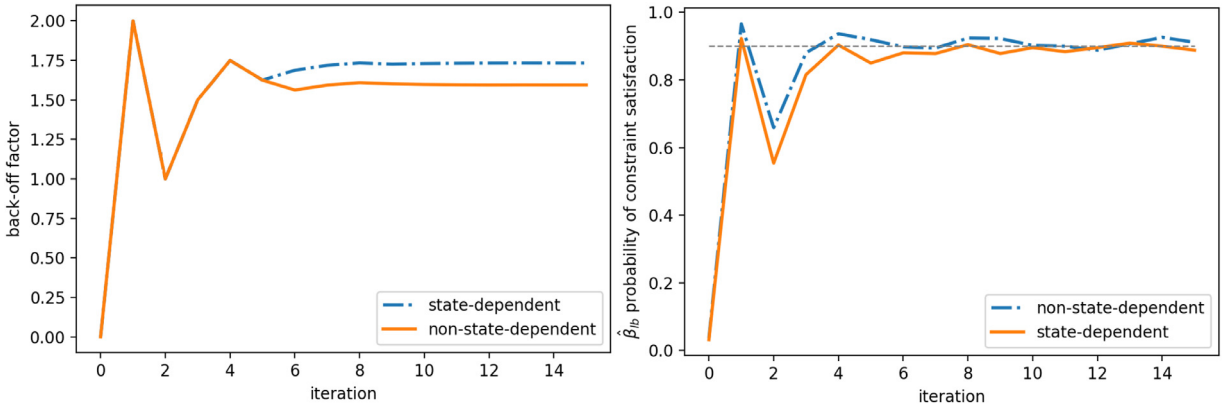

Fig. 7. Plots of evolution of the back-off factor and the probability of constraint satisfaction $\hat{\beta}_{lb}$ over the 16 back-off iterations for GP NMPC SD 50 and GP NMPC NSD 50.

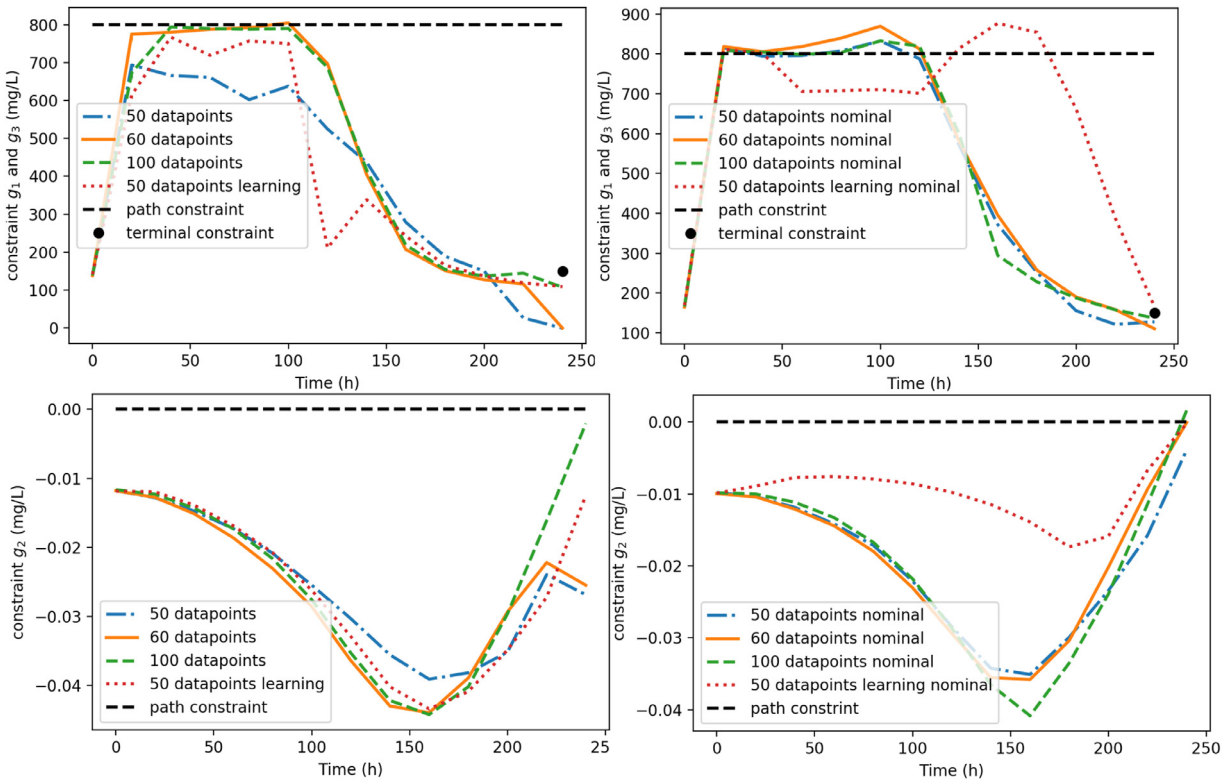

Fig. 8. Trajectories of the nitrate concentration constraints g_1 and g_3 (top) and the ratio of bioproduct to biomass constraint g_2 (bottom) for the GP NMPC 50, 60, 100, and GP NMPC 50 learning applied to the "real" plant model with the final tightened constraint set on the LHS and with no back-off constraints on the RHS referred to as *nominal*.

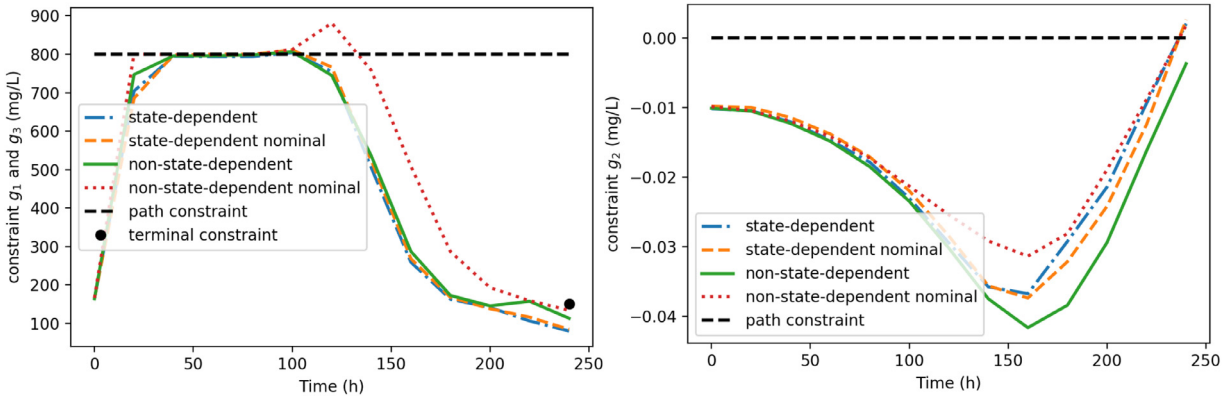

Fig. 9. Trajectories of the nitrate concentration constraints g_1 and g_3 (LHS) and the ratio of bioproduct to biomass constraint g_2 (RHS) for the GP NMPC 50, 60, 100, and GP NMPC 50 learning applied to the "real" plant model with the final tightened constraint set and with no back-off constraints referred to as *nominal*.

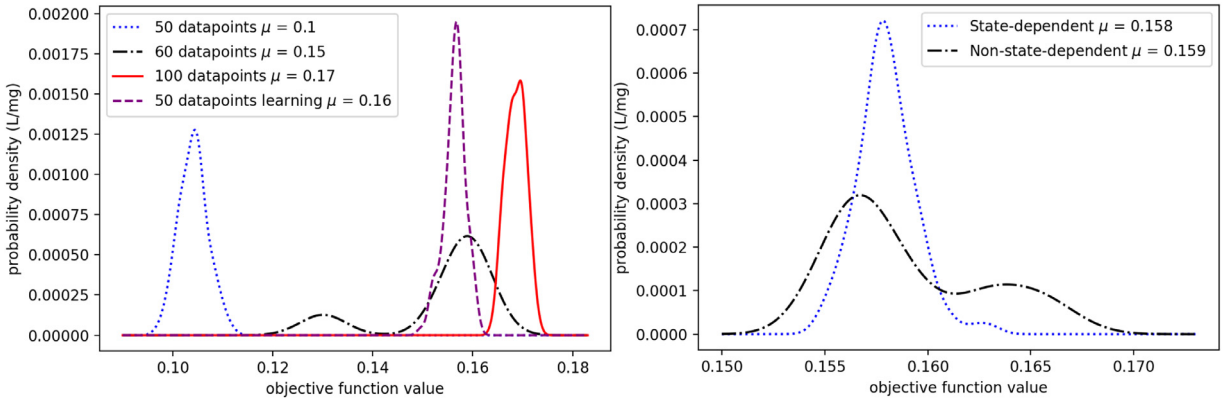

Fig. 10. Probability density function for the "real" plant objective values for all variations of the GP NMPC algorithm.

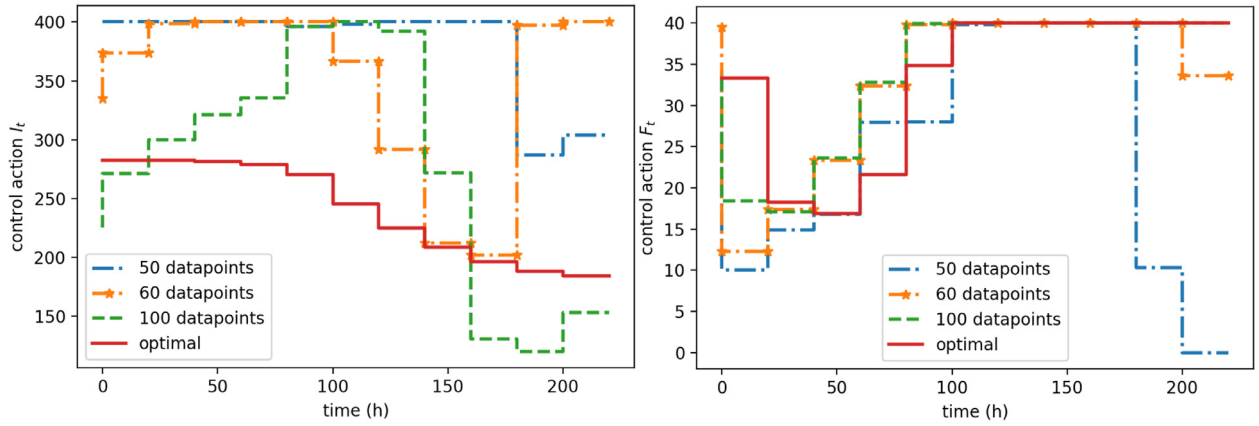

Fig. 11. Example control trajectories for the light intensity on the LHS and the nitrate flow rate on the RHS based on GP NMPC 50, 60, 100. The red line represents the optimal control trajectories ignoring the noise present in the process. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

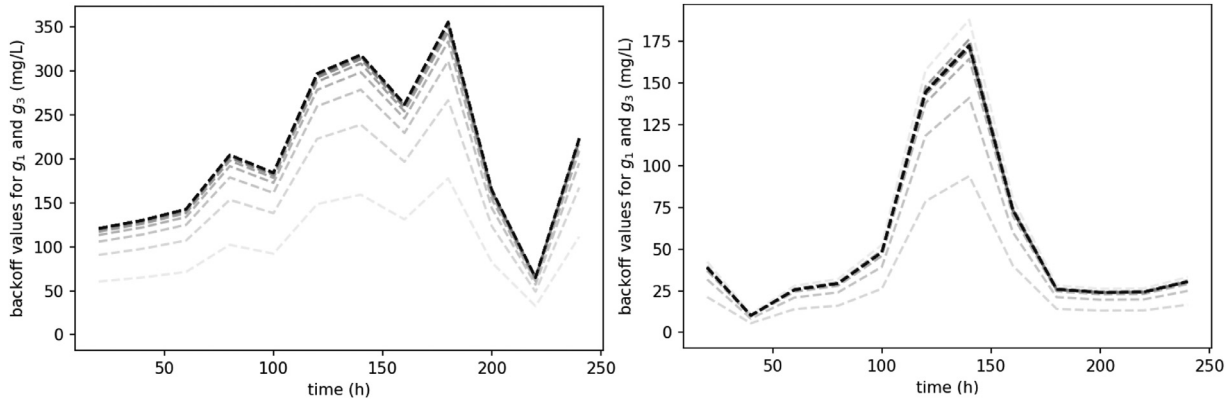

Fig. 12. Example back-off values for the nitrate concentration constraints g_1 and g_3 of GP NMPC 50 (LHS) and GP NMPC 50 learning (RHS). The lines are plotted over the 16 back-off iterations, which are faded out towards earlier iterations.

reached once over the 16 iterations, which is the case for all of them. Note that GP NMPC 50 does not converge, since even without back-offs the NMPC remains feasible for all MC trajectories and hence the bisection procedure fails. The performance of GP NMPC 50 is however also by far the worst. This is due to insufficient amounts of data in crucial areas for the control problem.

- From GP NMPC 50, 60 to 100 the objective values steadily increase and hence improve with increased number of data points as shown in Fig. 10. This is as expected, since more data points should lead to a more accurate GP plant model and hence more optimal control actions. Lastly, a more accurate GP plant model should also require less conservative back-offs. For the constraint g_2 the mean of the back-off values steadily decreases from 0.022 mg/L for GP NMPC 50 to 0.003 for GP NMPC 100 as shown in Table 2. For the constraints g_1/g_3 on the other hand the mean of the back-offs decreases dramatically from GP NMPC 50 with a value of 250mg/L to a value of 34.2mg/L for GP NMPC 60, while slightly increasing again for GP NMPC 100 to 38.8 mg/L. This is further illustrated in Fig. 4, for which the spread of the trajectories decreases steadily from GP NMPC 50, 60 to 100. All in all, larger datasets lead to improved solutions.
- The learning approach GP NMPC 50 learning leads to a reasonable solution with an objective value that is on average higher and therefore an improvement over GP NMPC 60 as can be seen in Fig. 10. Further, the sharper peak of the objective value suggests a more reliable performance. In contrast, GP NMPC 50 without learning is unable to determine a good solution and therefore has an objective value that is considerably worse than the remaining scenarios with an objective value that is on average over 30% lower. GP NMPC 50 is also unable to reach a $\hat{\beta}_{lb}$ value of 0.9 and has instead a much higher value as shown in Table 3, but at the expense of performance. This is highlighted in Fig. 11 in which the control trajectory of GP NMPC 50 decreases the nitrate flowrate early to satisfy the terminal constraint, which leads to a sub optimal solution. This is believed to be due to the high uncertainty of the $g_1/g_2/g_3$ constraint trajectories of GP NMPC 50, which can be seen by the large spread of the trajectories in Fig. 4. GP NMPC 50 learning on the other hand has a spread of the constraints $g_1/g_2/g_3$ that is significantly less than GP NMPC 50. This is further highlighted by the considerably higher back-off values of GP NMPC 50 compared to GP NMPC 50 learning as can be seen in Table 2, where the g_1/g_3 back-off values are nearly 400% larger, while the g_2 back-off values are over 300% larger. In conclusion, accounting for online learning has lead to a significantly better solution, although it should be noted that at larger datasets the effect is nearly negligible.
- GP NMPC 50 can be seen to be less erratic than GP NMPC 50 learning in Fig. 4, which is however expected. GP NMPC 50 uses the same prediction model throughout and hence the control inputs are only influenced by changes in the initial condition. For GP NMPC 50 learning on the other hand the prediction model changes at each sampling point and hence this leads to more irregular behaviour, which can be seen by the increased oscillations. This then leads to overall improved control actions

and performance due to exploiting the new data available, as can be seen in Fig. 10.

- It can be seen in Fig. 5 that GP NMPC NSD 50 has a larger spread of trajectories than GP NMPC SD 50. This is as expected, since GP NMPC SD 50 aims directly in its objective to minimize uncertainty. Consequently, the mean of the back-off values of constraints g_1/g_3 and g_2 in Table 2 are over 50% larger and over 100% greater than those for GP NMPC SD 50, respectively. Nonetheless, the attained average objective of GP NMPC SD 50 is marginally lower and hence worse as can be seen in Fig. 10. This is expected, since accounting for state dependency reduces conservativeness, but may also lead to a sub optimal solution due to conflicting objectives.
- In Table 3 the average computational times of a single GP NMPC evaluation are shown, which range from 135ms to 48ms. Overall, it can be seen that by far the largest computational times are attributed to GP NMPC 100, which is reasonable since the complexity of GP plant models grows exponentially with the number of data points. GP NMPC 50 and GP NMPC 50 learning can be seen however to have higher computational times, which is due to a more complex optimization problem from the reduced amount of data. This is further highlighted by GP NMPC 50 SD and GP NMPC 50 NSD attaining the lowest average computational times due to the second type of dataset leading to easier optimization solutions. In Table 3 we can further see that the computational times required for a single back-off iteration is for all variations nearly solely determined by the GP NMPC evaluation time. In addition, in Table 3 we show the ecdf value $\hat{\beta}$ for the final back-off iteration. It can be seen that these probabilities are substantially higher than the required probability of 0.9 ranging from 0.91 to 0.93 for the converged solutions, which is due to the conservativeness of the probabilistic lower bound used. This leads to higher back-off values than required and therefore worse objective values. This conservativeness can be reduced by setting α to a higher value.
- Lastly, in Figs. 8–9 trajectories of the constraints are shown by applying the GP NMPC variants to the "real" plant. It can be seen that the *nominal* variations of GP NMPC 50, 60, 100, and GP NMPC 50 learning with back-offs set to zero violate the nitrate constraint g_1 to remain below 800mg/L by a substantial amount of up to 50mg/L for GP NMPC 50 learning and GP NMPC 60. With back-offs on the other hand the approaches remain feasible throughout the run, which illustrates the importance of employing back-offs. GP NMPC NSD 50 *nominal* can be also seen to violate the nitrate constraint g_1 by 50mg/L, while GP NMPC SD 50 *nominal* does not violate this constraint. This is likely due to GP NMPC SD 50 *nominal* following a feasible trajectory in the dataset. GP NMPC NSD 50 with back-offs remains feasible. Overall, it can be seen that back-offs are important to achieve feasibility given the presence of plant-model mismatch.

7. Conclusions

In conclusion, a new approach is proposed for finite-horizon control problems using NMPC in conjunction with GP state space models. The method utilizes the probabilistic nature of GPs to sample deterministic functions of possible plant models. Tightened constraints using explicit back-offs are then determined, such that the closed-loop simulations of these possible plant models are feasible to a high probability. In addition, it is shown how probabilistic guarantees can be derived based on the number of constraint violations from the simulations. Furthermore, it is shown that online learning and state dependency of the uncertainty can be taken into account explicitly in this method, which leads to overall less conservativeness. Moreover, the computational times are shown to be relatively low, since constraint tightening is performed offline. Finally, through the comprehensive semi-batch bioprocess case study, the efficiency and potential of this method for the optimisation of complex stochastic systems (e.g. biological processes) is well demonstrated.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Eric Bradford: Conceptualization, Software, Writing - original draft, Formal analysis, Methodology, Validation. **Lars Imsland:** Supervision, Writing - review & editing. **Dongda Zhang:** Writing - review & editing, Methodology. **Ehecatl Antonio del Rio Chanona:** Conceptualization, Software, Writing - review & editing.

Acknowledgements

This project has received funding from the European Unions Horizon 2020 research and innovation programme under the Marie Sklodowska-Curie grant agreement No 675215.

Appendix A. Recursive inverse matrix update

In this paper we are often concerned with inverting matrices as shown in Section 3.2 in Eq. (17c):

$$\boldsymbol{\Sigma}_{\mathbf{Y}}^{+-1} = \begin{bmatrix} \boldsymbol{\Sigma}_{\mathbf{Y}} & \mathbf{k}^{\mathrm{T}}(\mathbf{z}^{+}) \\ \mathbf{k}(\mathbf{z}^{+}) & k(\mathbf{z}^{+},\mathbf{z}^{+})(+\sigma_{\nu}^{2}) \end{bmatrix}^{-1} \tag{A.1}$$

This is an recursive update formula and hence $\boldsymbol{\Sigma}_{\mathbf{Y}}^{-1}$ is known *a priori*. In this Section we introduce a recursive formula to exploit this fact to obtain $\boldsymbol{\Sigma}_{\mathbf{Y}}^{+-1}$ in a cheaper fashion taken from Strassen (1969) adjusted to our case. The following quantities need to be computed to obtain $\boldsymbol{\Sigma}_{\mathbf{Y}}^{+-1}$:

$$\mathrm{I} = \mathbf{k}^{\mathrm{T}}(\mathbf{z}^{+})\boldsymbol{\Sigma}_{\mathbf{Y}}^{-1} \tag{A.2a}$$

$$\mathrm{II} = \mathbf{k}^{\mathrm{T}}(\mathbf{z}^{+})\mathrm{I}^{\mathrm{T}} \tag{A.2b}$$

$$\mathbf{C}_{12} = \mathrm{I}^{\mathrm{T}} \times \mathrm{II} \tag{A.2c}$$

$$C_{11} = \boldsymbol{\Sigma}_{\mathbf{Y}}^{-1} - \mathrm{I}^{\mathrm{T}} \times \mathbf{C}_{12}^{\mathrm{T}} \tag{A.2d}$$

$$C_{22} = -\left(\mathrm{II} - k(\mathbf{z}^{+},\mathbf{z}^{+})(+\sigma_{\nu}^{2})\right)^{-1} \tag{A.2e}$$

T269 The inverted matrix is then given by:

$$\boldsymbol{\Sigma}_{\mathbf{Y}}^{+-1} = \begin{bmatrix} C_{11} & \mathbf{C}_{12} \\ \mathbf{C}_{12}^{\mathrm{T}} & C_{22} \end{bmatrix} \tag{A.3}$$

References

Aiba, S., 1982. Growth Kinetics of Photosynthetic Microorganisms. In: Microbial reactions. In: Advances in Biochemical Engineering, vol. 23. Springer, pp. 85–156.
Alamo, T., Tempo, R., Luque, A., Ramirez, D.R., 2015. Randomized methods for design of uncertain systems: sample complexity and sequential algorithms. Automatica 52, 160–172.
Allgöwer, F., Findeisen, R., Nagy, Z.K., 2004. Nonlinear model predictive control: from theory to application. J. Chin. Inst. Chem. Eng, 35 (3), 299–315.
Andersson, J.A.E., Gillis, J., Horn, G., Rawlings, J.B., Diehl, M., 2019. CasADi: a software framework for nonlinear optimization and optimal control. Mathematical Programming Computation 11 (1), 1–36.
Beers, K.J., 2007. Numerical methods for chemical engineering: applications in matlab. Cambridge University Press.

Bemporad, A., Morari, M., 1999. Robust Model Predictive Control: A Survey. In: Robustness in identification and control. In: Lecture Notes in Control and Information Sciences, 245. Springer, pp. 207–226.

Berkenkamp, F., Schoellig, A.P., 2015. Safe and robust learning control with Gaussian processes. In: European control conference (ECC). IEEE, pp. 2496–2501.

Biegler, L.T., 2010. Nonlinear programming: Concepts, algorithms, and applications to chemical processes. MOS-SIAM Series on Optimization. Society for Industrial & Applied Mathematics.

Biegler, L.T., Rawlings, J.B., 1991. Optimization approaches to nonlinear model predictive control. Technical Report.

Bradford, E., Imsland, L., 2018a. Economic stochastic model predictive control using the unscented kalman filter. IFAC-PapersOnLine 51 (18), 417–422.

Bradford, E., Imsland, L., 2018b. Stochastic Nonlinear Model Predictive Control Using Gaussian Processes. In: European control conference (ECC). IEEE, pp. 1027–1034.

Bradford, E., Imsland, L., 2019. Output feedback stochastic nonlinear model predictive control for batch processes. Computers & Chemical Engineering 126, 434–450.

Bradford, E., Imsland, L., del Rio-Chanona, E.A., 2019. Nonlinear model predictive control with explicit back-offs for gaussian process state space models. In: 58th conference on decision and control (CDC). IEEE, pp. 4747–4754.

Bradford, E., Schweidtmann, A.M., Lapkin, A., 2018a. Efficient multiobjective optimization employing gaussian processes, spectral sampling and a genetic algorithm. J. Global Optim. 71 (2), 407–438.

Bradford, E., Schweidtmann, A.M., Zhang, D., Jing, K., del Rio-Chanona, E.A., 2018b. Dynamic modeling and optimization of sustainable algal production with uncertainty using multivariate gaussian processes. Computers & Chemical Engineering 118, 143–158.

Cao, G., Lai, E.M.-K., Alam, F., 2017. Gaussian process model predictive control of an unmanned quadrotor. Journal of Intelligent & Robotic Systems 88 (1), 147–162.

Chowdhary, G., Kingravi, H.A., How, J.P., Vela, P.A., 2015. Bayesian nonparametric adaptive control using gaussian processes. IEEE Trans. Neural Netw. Learn. Syst. 26 (3), 537–550.

Clopper, C.J., Pearson, E.S., 1934. The use of confidence or fiducial limits illustrated in the case of the binomial. Biometrika 26 (4), 404–413.

Conti, S., Gosling, J.P., Oakley, J.E., O'Hagan, A., 2009. Gaussian process emulation of dynamic computer codes. Biometrika 96 (3), 663–676.

Curtis, F.E., Wachter, A., Zavala, V.M., 2018. A sequential algorithm for solving nonlinear optimization problems with chance constraints. SIAM J. Optim. 28 (1), 930–958.

Deisenroth, M., Rasmussen, C.E., 2011. PILCO: A model-based and data-efficient approach to policy search. In: Proceedings of the 28th international conference on machine learning (ICML-11), pp. 465–472.

Girard, A., Rasmussen, C.E., Candela, J.Q., Murray-Smith, R., 2003. Gaussian Process Priors with Uncertain Inputs Application to Multiple-step Ahead Time Series Forecasting. In: Advances in neural information processing systems 15. MIT Press, pp. 545–552.

Grancharova, A., Kocijan, J., Johansen, T.A., 2007. Explicit stochastic nonlinear predictive control based on Gaussian process models. In: European control conference (ECC). IEEE, pp. 2340–2347.

Heirung, T.A.N., Paulson, J.A., O'Leary, J., Mesbah, A., 2018. Stochastic model predictive controlhow does it work? Computers & Chemical Engineering 114, 158–170.

Hewing, L., Kabzan, J., Zeilinger, M.N., 2019. Cautious model predictive control using gaussian process regression. IEEE Trans. Control Syst. Technol. 1–8.

Hewing, L., Liniger, A., Zeilinger, M.N., 2018. Cautious NMPC with Gaussian Process Dynamics for Autonomous Miniature Race Cars. In: European control conference (ECC), pp. 1341–1348.

Hindmarsh, A.C., Brown, P.N., Grant, K.E., Lee, S.L., Serban, R., Shumaker, D.E., Woodward, C.S., 2005. SUNDIALS: Suite of nonlinear and differential/algebraic equation solvers. ACM Transactions on Mathematical Software (TOMS) 31 (3), 363–396.

Kavsek-Biasizzo, K., Skrjanc, I., Matko, D., 1997. Fuzzy predictive control of highly nonlinear pH process. Computers & chemical engineering 21, S613–S618.

Klenske, E.D., Zeilinger, M.N., Schölkopf, B., Hennig, P., 2016. Gaussian process-based predictive control for periodic error correction. IEEE Trans. Control Syst. Technol. 24 (1), 110–121.

Kocijan, J., Girard, A., Banko, B., Murray-Smith, R., 2005. Dynamic systems identification with gaussian processes. Math. Comput. Model Dyn. Syst. 11 (4), 411–424.

Kocijan, J., Murray-Smith, R., 2005. Nonlinear Predictive Control with a Gaussian Process Model. In: Switching and learning in feedback systems. Springer, pp. 185–200.

Kocijan, J., Murray-Smith, R., Rasmussen, C.E., Girard, A., 2004. Gaussian process model based predictive control. In: American control conference (ACC), vol. 3. IEEE, pp. 2214–2219.

Koller, R.W., RicardezSandoval, L.A., Biegler, L.T., 2018a. Stochastic backoff algorithm for simultaneous design, control, and scheduling of multiproduct systems under uncertainty. AIChE J. 64 (7), 2379–2389.

Koller, T., Berkenkamp, F., Turchetta, M., Krause, A., 2018b. Learning-based model predictive control for safe exploration. In: 57th conference on decision and control (CDC). IEEE, pp. 6059–6066.

Krige, D.G., 1951. A statistical approach to some basic mine valuation problems on the witwatersrand. J. South. Afr. Inst. Min. Metall 52 (6), 119–139.

Likar, B., Kocijan, J., 2007. Predictive control of a gasliquid separation plant based on a gaussian process model. Computers & chemical engineering 31 (3), 142–152.

Lucia, S., 2014. Robust multi-stage nonlinear model predictive control. Schriftenreihe des Lehrstuhls fuer Systemdynamik und Prozessfuehrung. Shaker.

Lucia, S., Finkler, T., Engell, S., 2013. Multi-stage nonlinear model predictive control applied to a semi-batch polymerization reactor under uncertainty. J. Process. Control 23 (9), 1306–1319.

Maciejowski, J.M., 2002. Predictive control with constraints. Pearson education.

Maciejowski, J.M., Yang, X., 2013. Fault tolerant control using Gaussian processes and model predictive control. In: Control and fault-tolerant systems (systol). IEEE, pp. 1–12.

Maiworm, M., Limon, D., Manzano, J.M., Findeisen, R., 2018. Stability of gaussian process learning based output feedback model predictive control. IFAC-PapersOnLine 51 (20), 455–461.

Mesbah, A., 2016. Stochastic model predictive control: an overview and perspectives for future research. IEEE Control Syst. Mag. 36 (6), 30–44.

Mesbah, A., Streif, S., Findeisen, R., Braatz, R.D., 2014. Stochastic nonlinear model predictive control with probabilistic constraints. In: American control conference (ACC). IEEE, pp. 2413–2419.

Murray-Smith, R., Sbarbaro, D., Rasmussen, C.E., Girard, A., 2003. Adaptive, cautious, predictive control with gaussian process priors. IFAC Proceedings Volumes 36 (16), 1155–1160.

Nagy, Z.K., Braatz, R.D., 2003. Robust nonlinear model predictive control of batch processes. AIChE J. 49 (7), 1776–1786.

Nagy, Z.K., Mahn, B., Franke, R., Allgöwer, F., 2007a. Evaluation study of an efficient output feedback nonlinear model predictive control for temperature tracking in an industrial batch reactor. Control Eng. Pract. 15 (7), 839–850.

Nagy, Z.K., Mahn, B., Franke, R., Allgöwer, F., 2007b. Real-time Implementation of Nonlinear Model Predictive Control of Batch Processes in an Industrial Framework. In: Assessment and future directions of nonlinear model predictive control. Springer, pp. 465–472.

Paulson, J.A., Mesbah, A., 2018. Nonlinear model predictive control with explicit backoffs for stochastic systems under arbitrary uncertainty. IFAC-PapersOnLine 51 (20), 523–534.

Paulson, J.A., Mesbah, A., 2019. An efficient method for stochastic optimal control with joint chance constraints for nonlinear systems. Int. J. Robust Nonlinear Control 29 (15), 5017–5037.

Piche, S., Sayyar-Rodsari, B., Johnson, D., Gerules, M., 2000. Nonlinear model predictive control using neural networks. IEEE Control Syst. Mag. 20 (3), 53–62.

Quionero-Candela, J., Rasmussen, C.E., Figueiras-Vidal, A.A.R., 2010. Sparse spectrum gaussian process regression. Journal of Machine Learning Research 11 (Jun), 1865–1881.

Rasmussen, C.E., Williams, C.K.I., 2006. Gaussian processes for machine learning. Adaptive Computation and Machine Learning. The MIT Press.

del Rio-Chanona, E.A., Dechatiwongse, P., Zhang, D., Maitland, G.C., Hellgardt, K., Arellano-Garcia, H., Vassiliadis, V.S., 2015. Optimal operation strategy for biohydrogen production. Industrial & Engineering Chemistry Research 54 (24), 6334–6343.

Sampson, P.D., Guttorp, P., 1992. Nonparametric estimation of nonstationary spatial covariance structure. J. Am. Stat. Assoc. 87 (417), 108–119.

Sobol, I.M., 2001. Global sensitivity indices for nonlinear mathematical models and their monte carlo estimates. Math. Comput. Simul. 55 (1–3), 271–280.

Soloperto, R., Müller, M.A., Trimpe, S., Allgöwer, F., 2018. Learning-Based robust model predictive control with state-Dependent uncertainty. IFAC-PapersOnLine 51 (20), 442–447.

Strassen, V., 1969. Gaussian elimination is not optimal. Numerische mathematik 13 (4), 354–356.

Streif, S., Karl, M., Mesbah, A., 2014. Stochastic nonlinear model predictive control with efficient sample approximation of chance constraints. arXiv preprint arXiv:1410.4535.

Sun, Z., Qin, S.J., Singhal, A., Megan, L., 2013. Performance monitoring of model-predictive controllers via model residual assessment. J. Process. Control 23 (4), 473–482.

Umlauft, J., Beckers, T., Hirche, S., 2018. Scenario-based Optimal Control for Gaussian Process State Space Models. In: European control conference (ECC). IEEE, pp. 1386–1392.

Umlauft, J., Beckers, T., Kimmel, M., Hirche, S., 2017. Feedback linearization using Gaussian processes. In: 56th conference on decision and control (CDC). IEEE, pp. 5249–5255.

Valappil, J., Georgakis, C., 2002. Nonlinear model predictive control of enduse properties in batch reactors. AIChE J. 48 (9), 2006–2021.

Wächter, A., Biegler, L.T., 2006. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. Math. Program. 106 (1), 25–57.

Wang, Y., Ocampo-Martinez, C., Puig, V., 2016. Stochastic model predictive control based on gaussian processes applied to drinking water networks. IET Control Theory & Applications 10 (8), 947–955.

Wu, Z., Rincon, D., Christofides, P.D., 2019a. Real-Time adaptive machine-Learning-Based predictive control of nonlinear processes. Industrial & Engineering Chemistry Research 59 (6), 2275–2290.

Wu, Z., Tran, A., Rincon, D., Christofides, P.D., 2019b. Machine learning based predictive control of nonlinear processes. part i: theory. AIChE J. 65 (11), e16729.

Wu, Z., Tran, A., Rincon, D., Christofides, P.D., 2019c. Machine learning based predictive control of nonlinear processes. part II: computational implementation. AIChE J. 65 (11), e16734.

Xi, X.-C., Poo, A.-N., Chou, S.-K., 2007. Support vector regression model predictive control on a HVAC plant. Control Eng. Pract. 15 (8), 897–908.